

Digital Design

Register and SRAM Memory with Verilog Model

A. Sahu

Dept of Comp. Sc. & Engg.

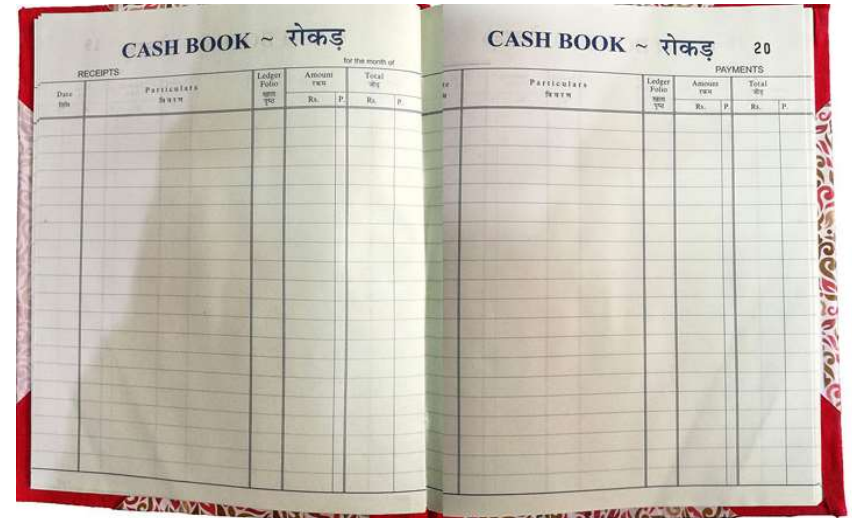
Indian Institute of Technology Guwahati

**Some part of Slides Adopted from Digital System Book
by Frank Vahid**

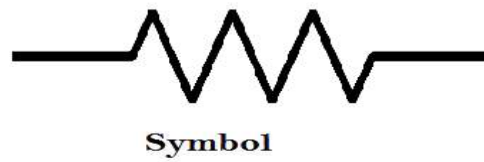
Outline

- Register
- Shift register
- Universal Register
- Multi-function Register
- Memory
- Counter
- Sync Counter
- Digital Clock : Case Study

Register

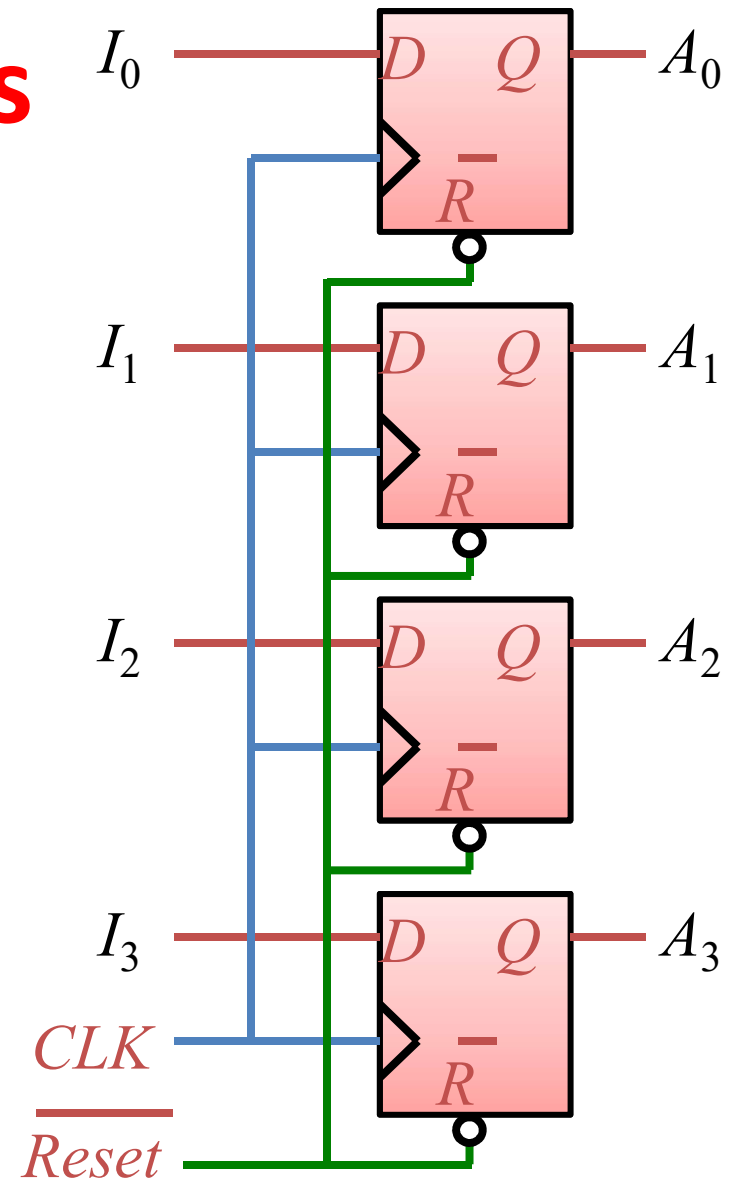


not Resistor

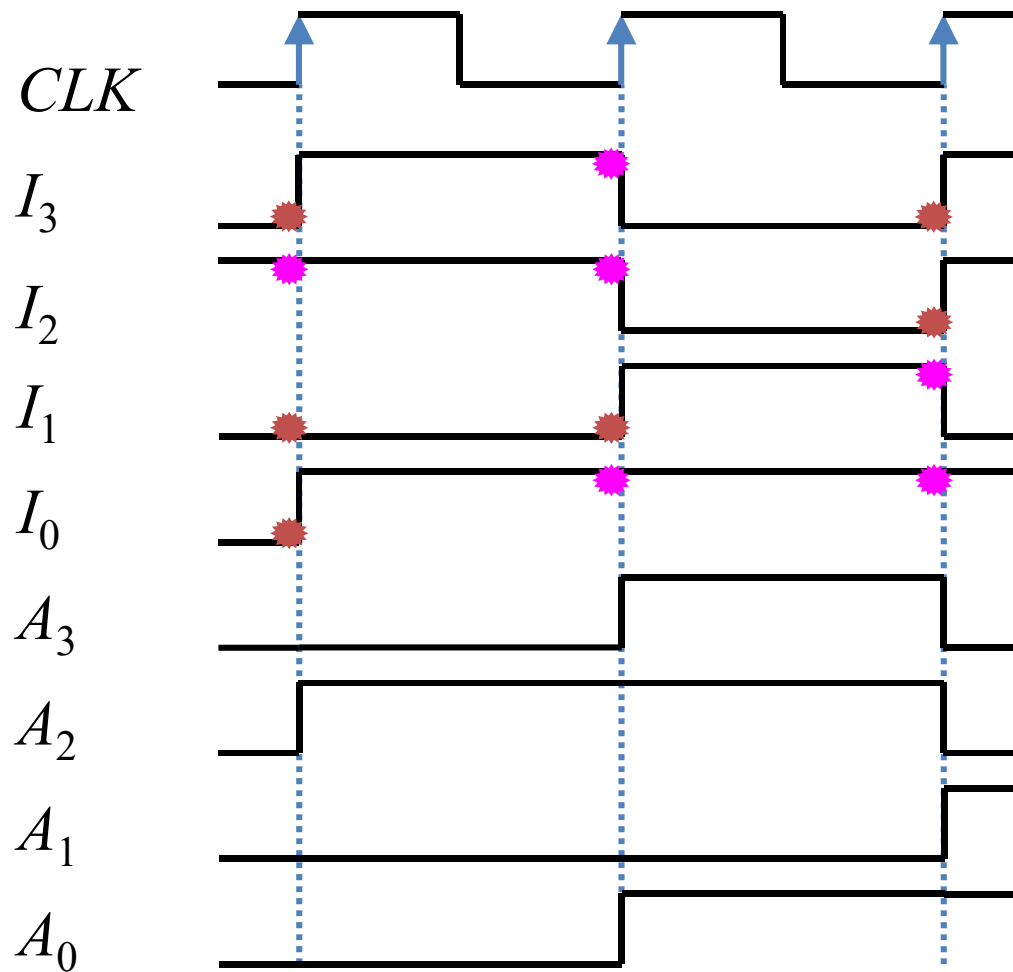


Registers

- Group of D Flip-Flops
- Synchronized (Single Clock)
- Store Data

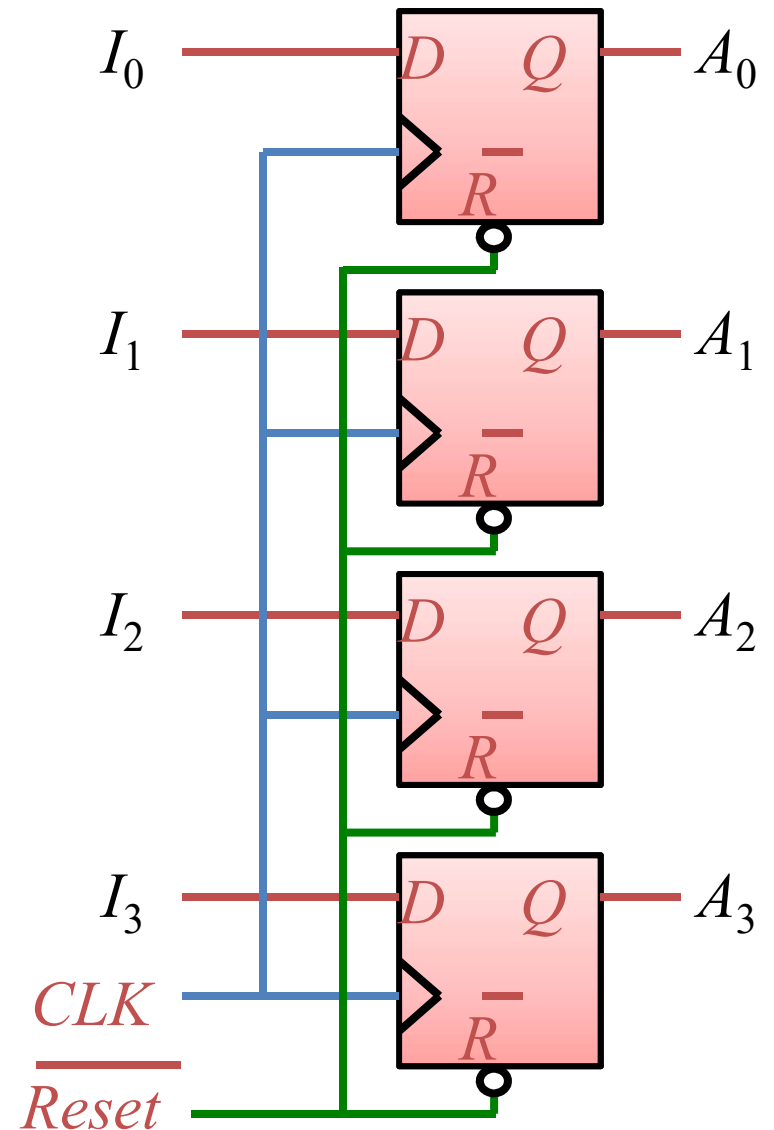


Registers



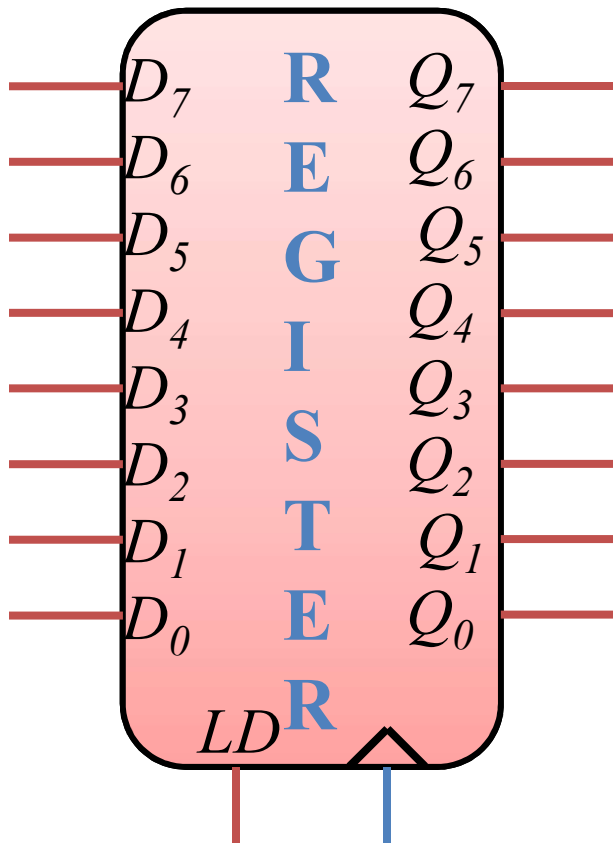
Note: New data has to go in with every clock

**See carefully: Input at the dotted will be reflected to output :
Just before rising edges**



Registers with Parallel Load

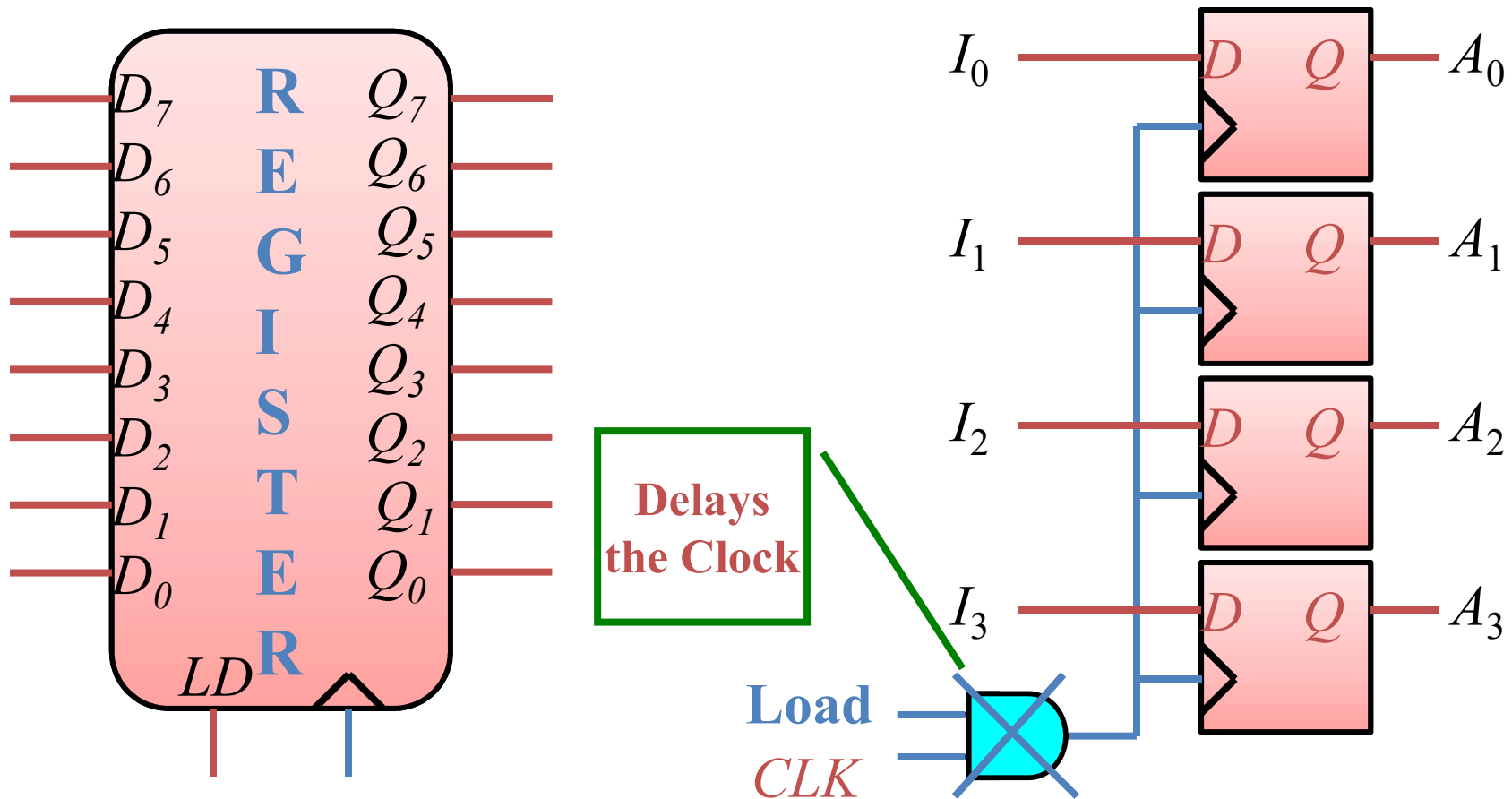
- Control **Loading** the Register with New Data



LD	$Q(t+1)$
0	$Q(t)$
1	D

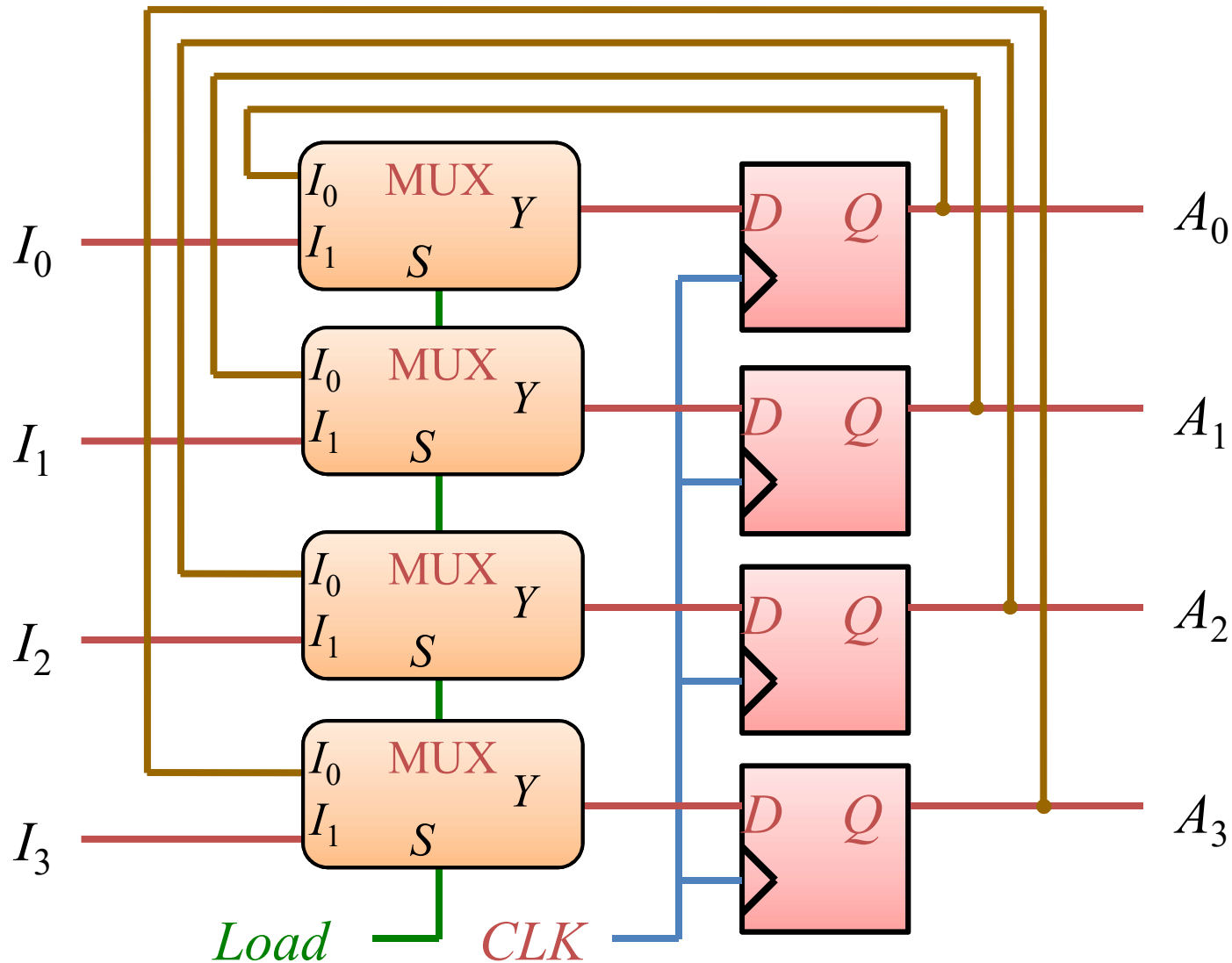
Registers with Parallel Load

- Should we block the “Clock” to keep the “Data”?



Registers with Parallel Load

- Circulate the “old data”



Shift Registers

- Register (Set of FFs)
- 4-Bit Shift Register (Example)
- Serial in Serial Out (SISO)

Shift Register

- Right Shift Example (Left shift is similar)
 - Move each bit one position right
 - Shift in 0 to leftmost bit

Q: Do four right shifts on 1001, showing value after each shift

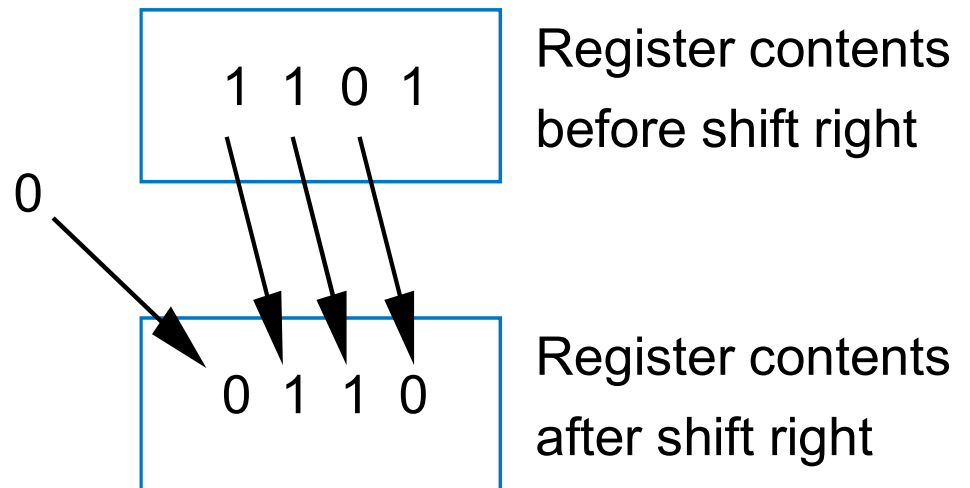
A: 1001 (original)

0100

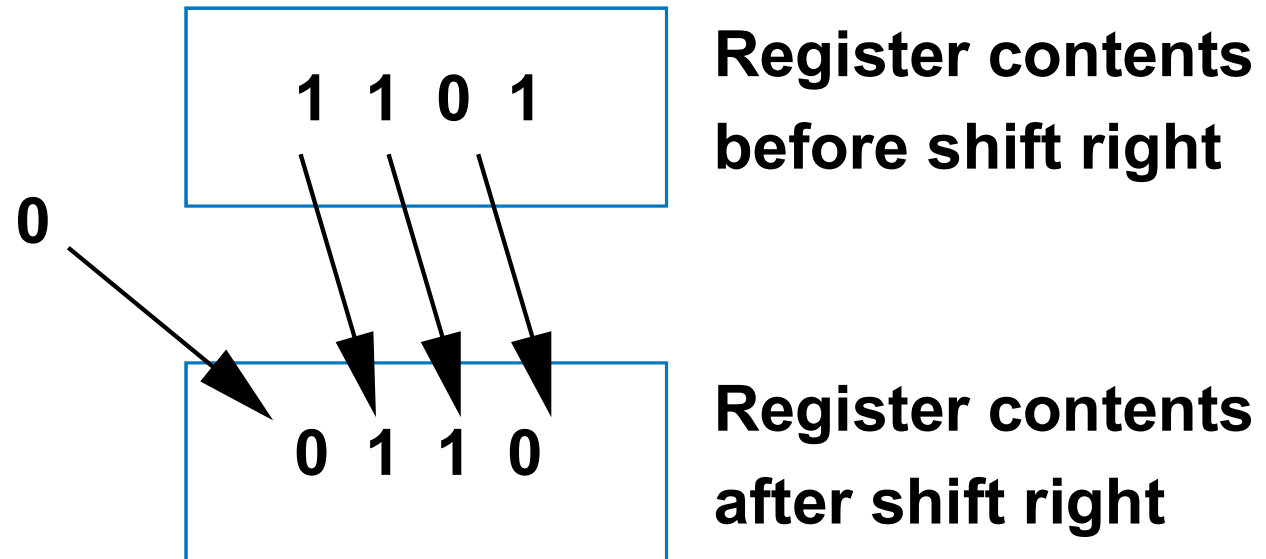
0010

0001

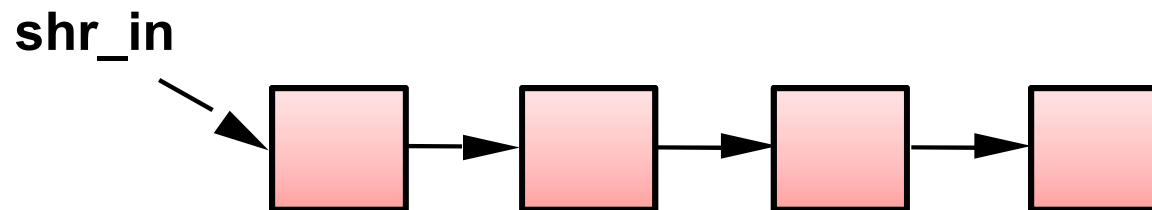
0000



Shift Register

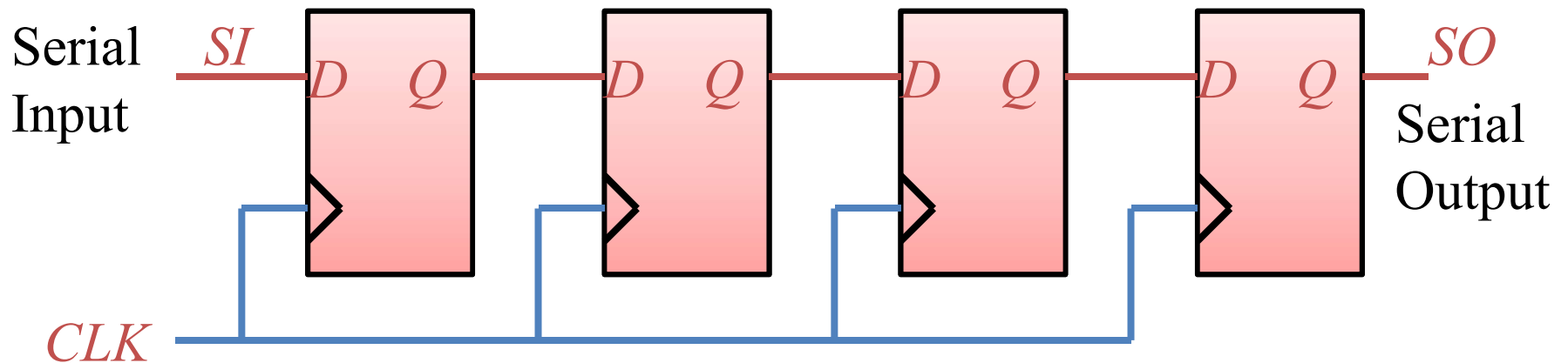


- **Implementation: Connect flip-flop output to next flip-flop's input**

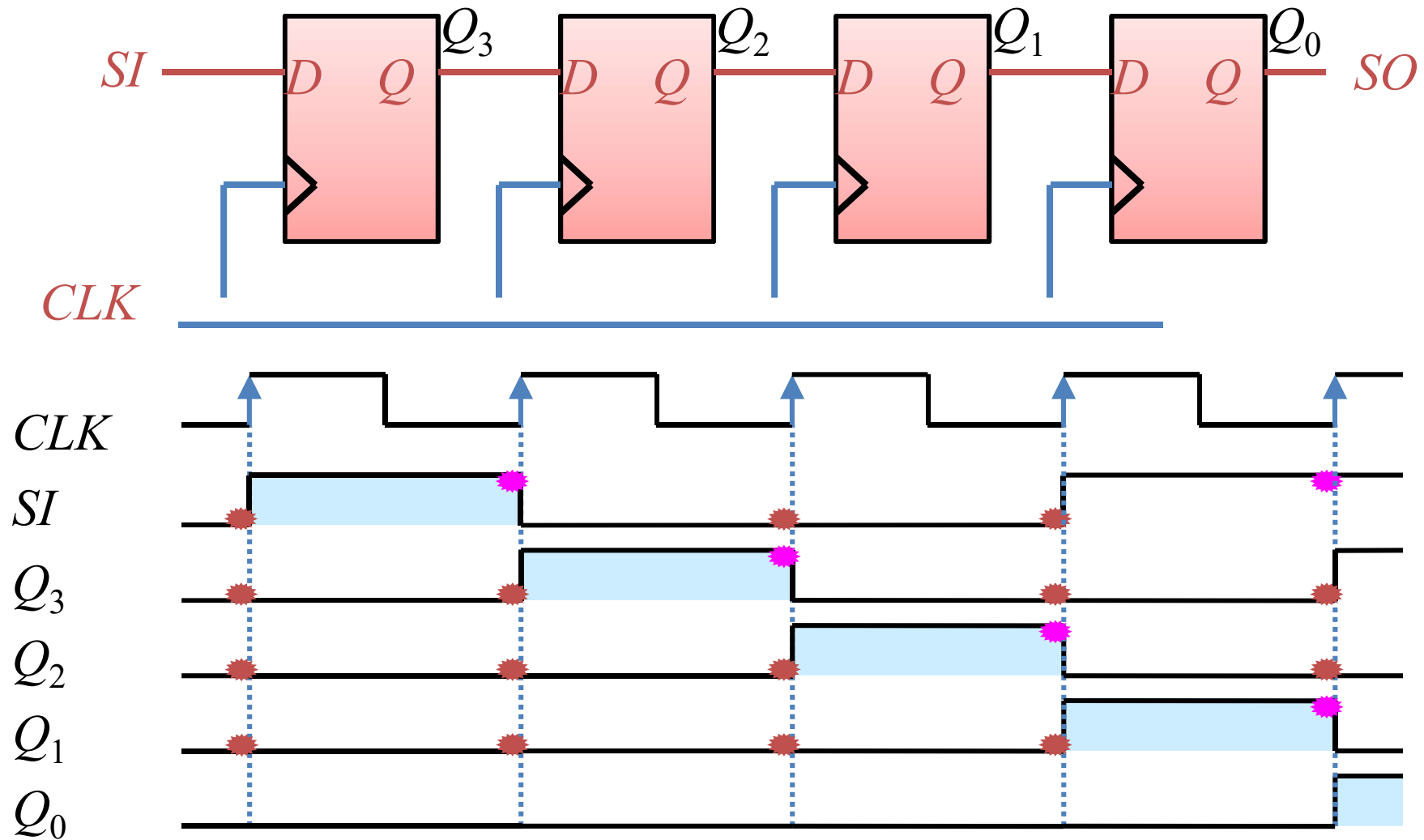


Shift Registers

- 4-Bit Shift Register
- Serial in Serial Out (SISO)

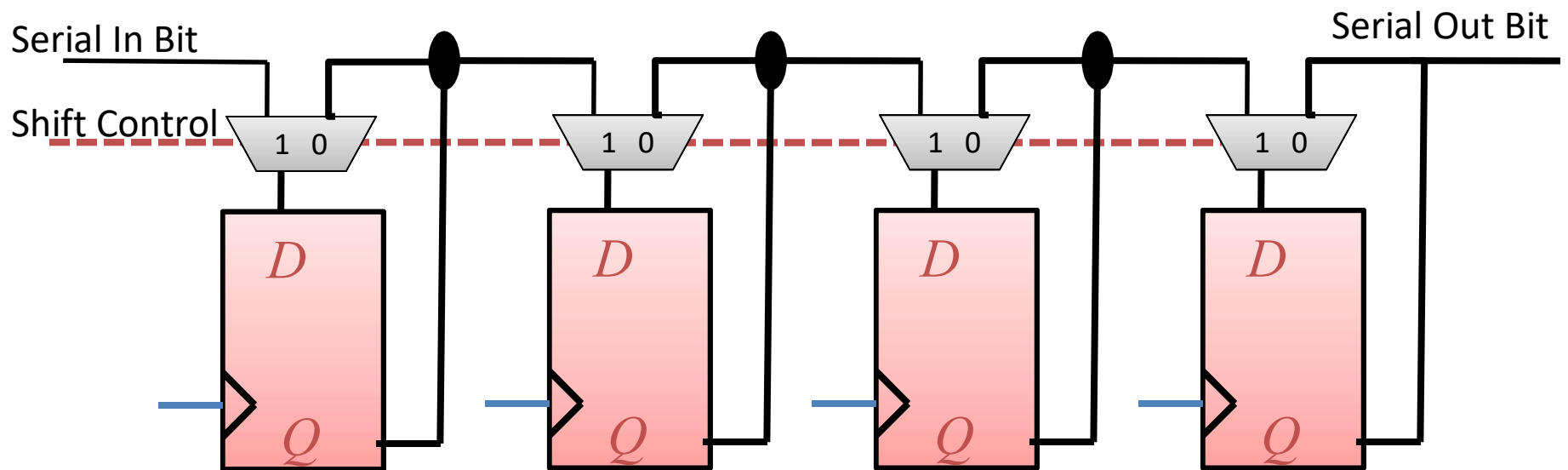


Shift Registers (SISO)



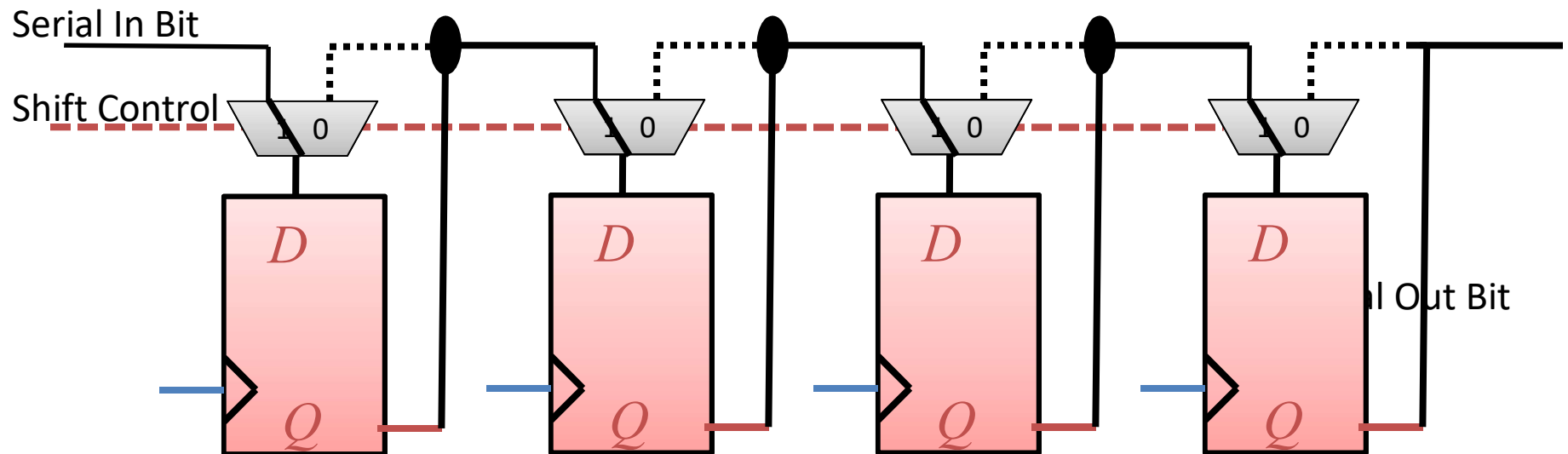
Shift Register with Control

- To allow register to either shift or retain, use 2x1 muxes
 - **Shift Control: shr: 0 means retain, 1 shift**
 - shr_in: value to shift in : May be 0/1, or right most bit



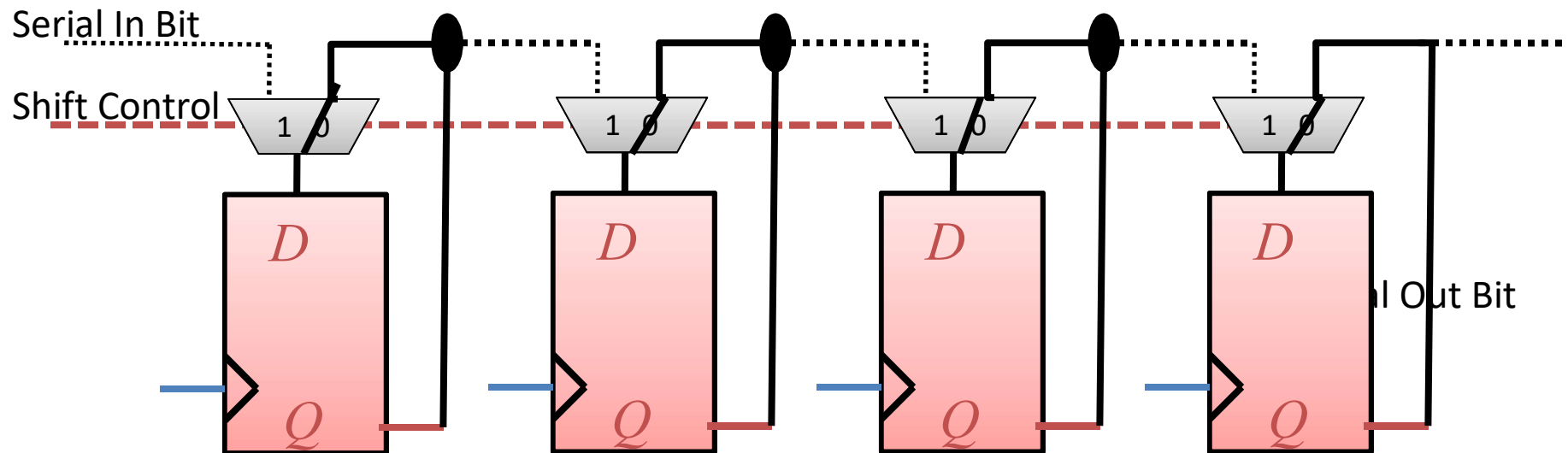
Shift Register with Control

Shift Control=1, Do one right shift per cycle

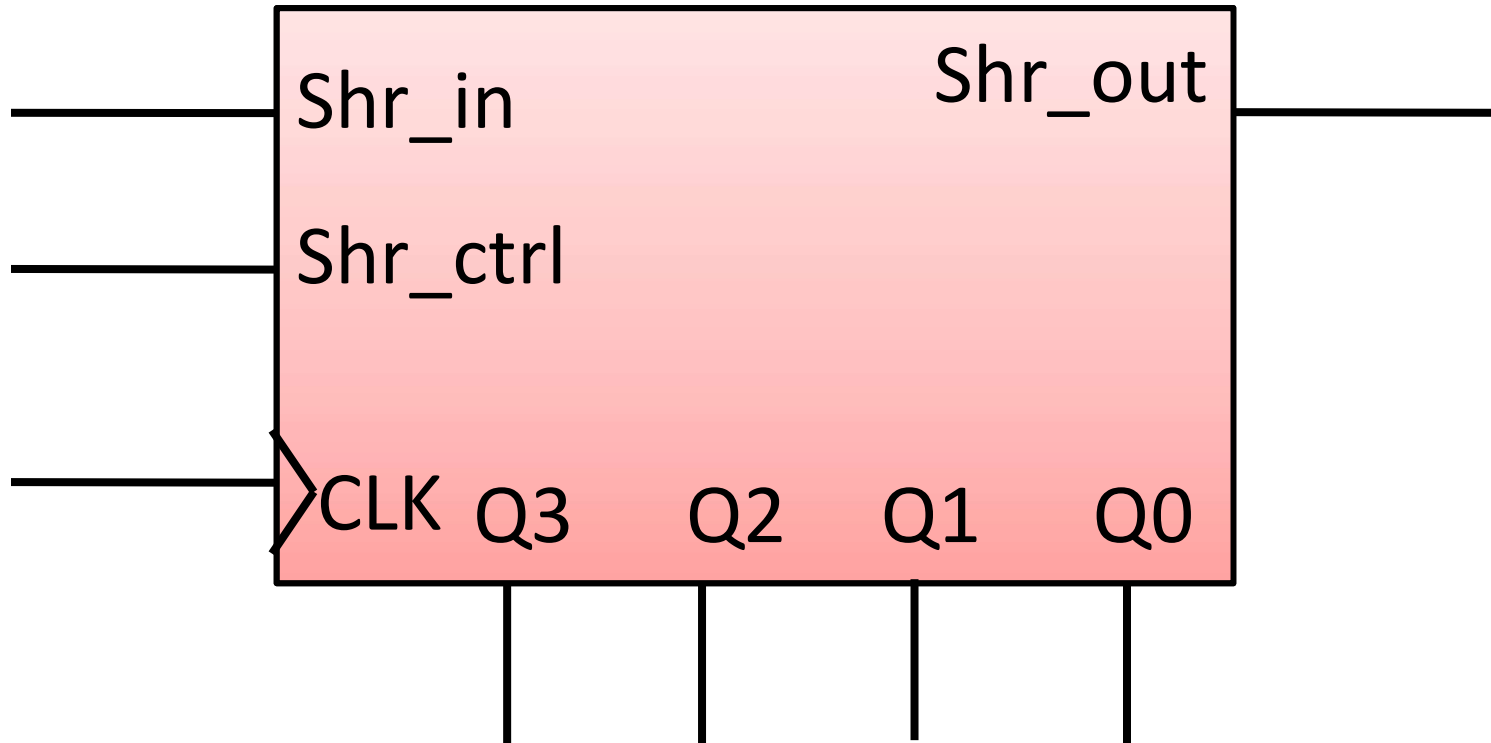


Shift Register with Control

Shift Control=0, No change to Data

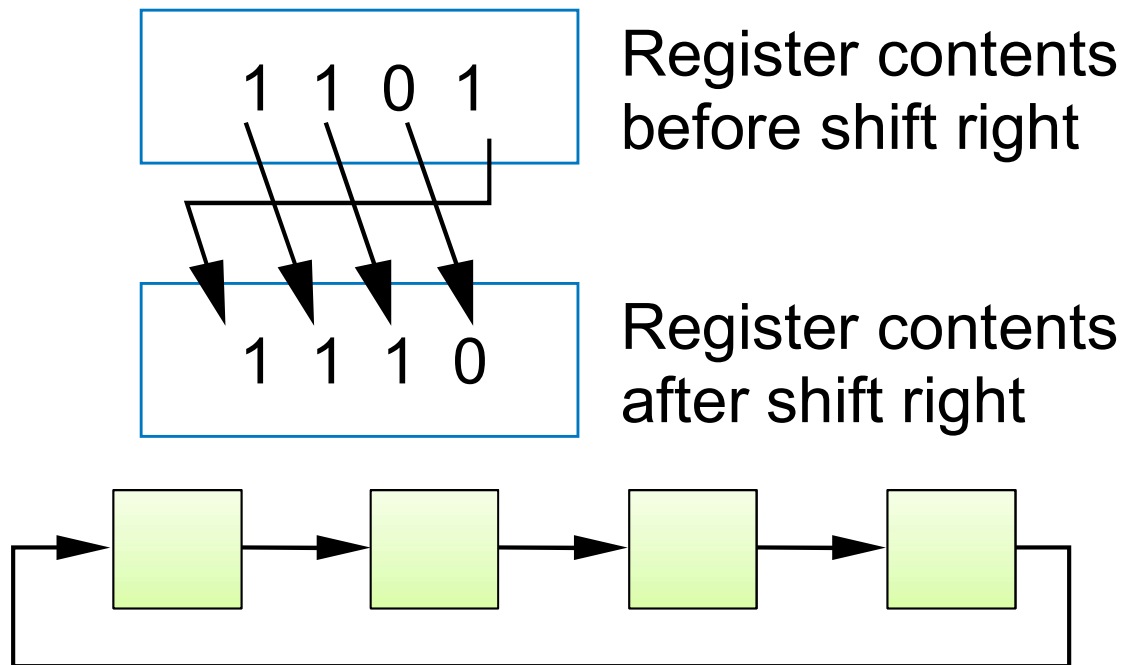


Shift Register with Control

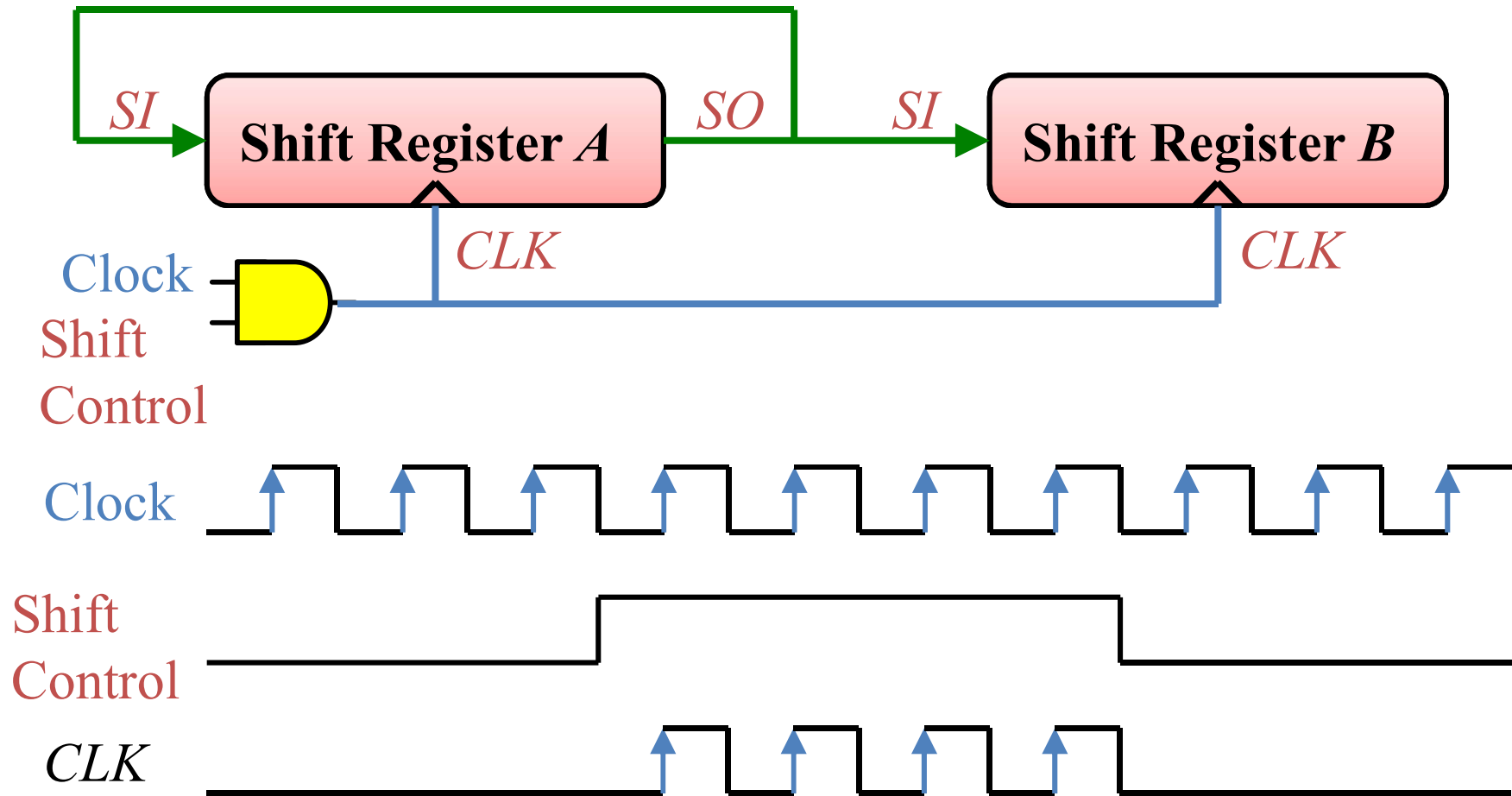


Rotate Register

- Rotate right: Like shift right, but leftmost bit comes from rightmost bit

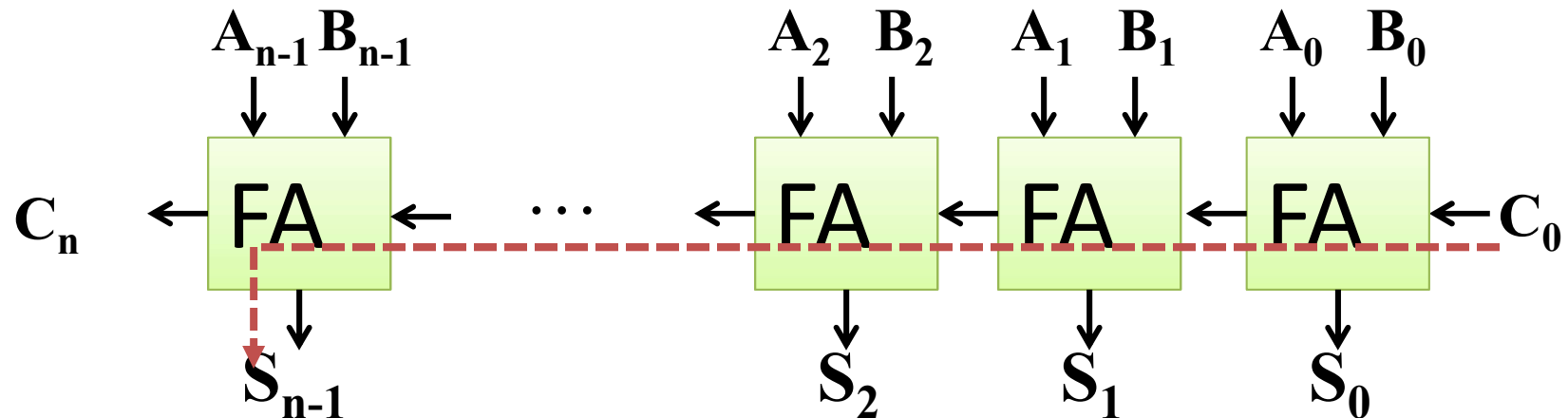


Serial Transfer

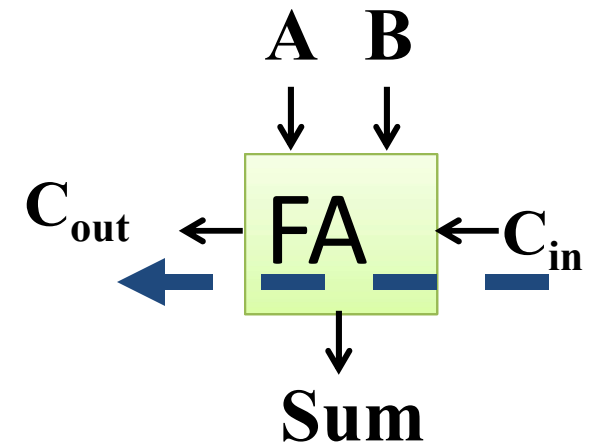


N-Bit RCA: Series of FA Cells

- To add two n-bit numbers

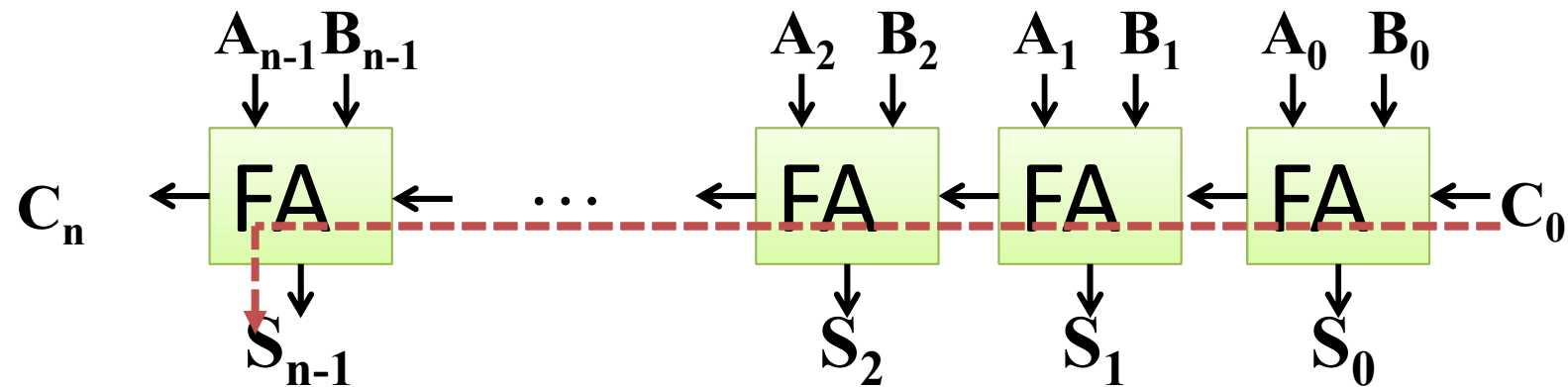


- Adder delay = $T_c * n$
- $T_c = (C_{in} \text{ to } C_{out} \text{ delay}) \text{ of a FA}$



N-Bit Ripple-Carry Adder

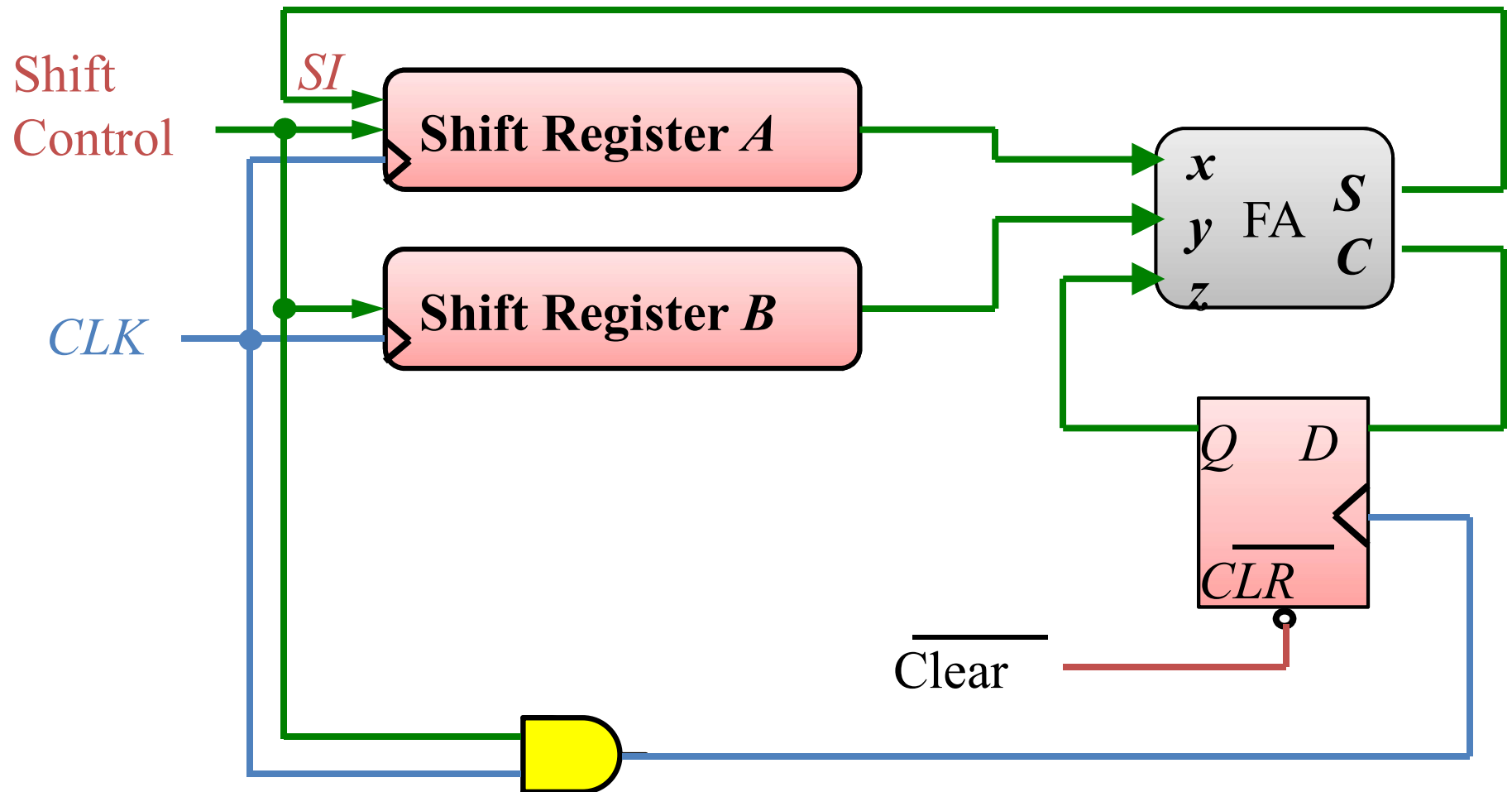
- To add two n-bit numbers



- Any point of time: only one FA is active

Do we require N Full Adder?

Serial Addition

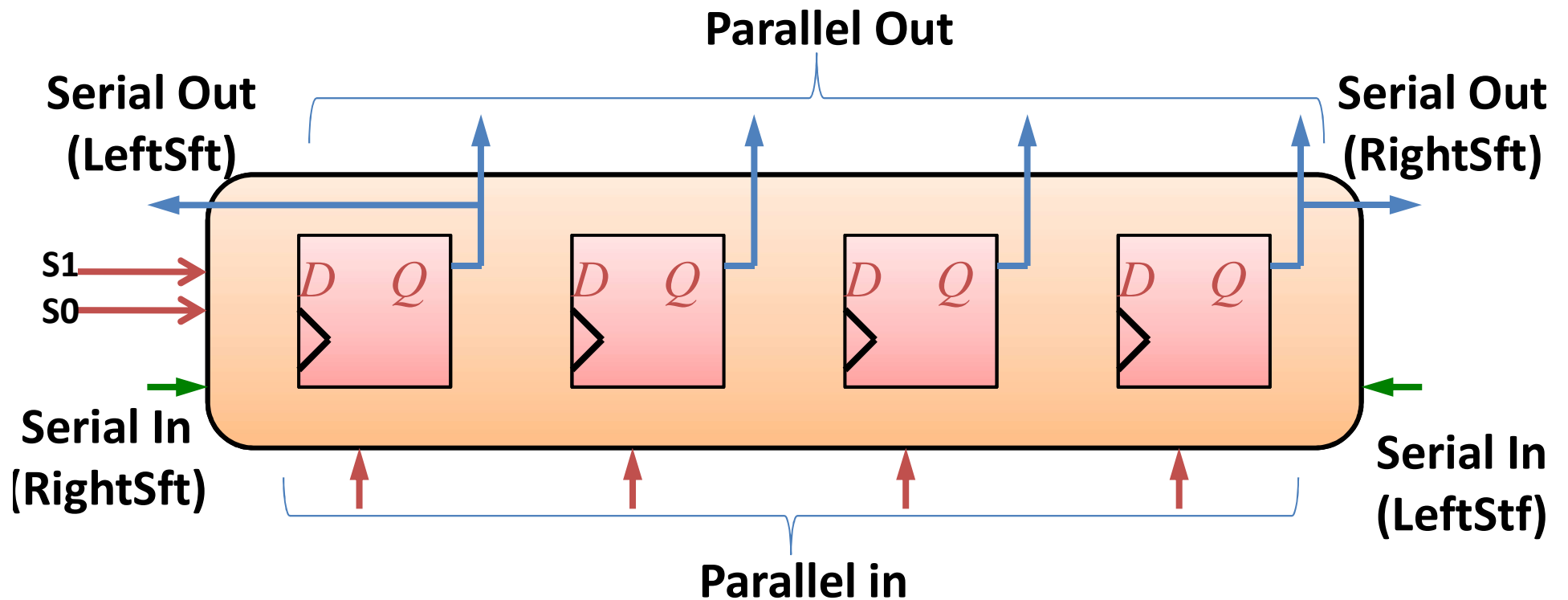


Operation Perform: $A = A + B$

Universal Shift Register

- Parallel-in Parallel-out (PIPO)
- Serial-in Serial-out (SISO)
- Serial-in Parallel-out (SIPO)
- Parallel-in Serial-out (PISO)

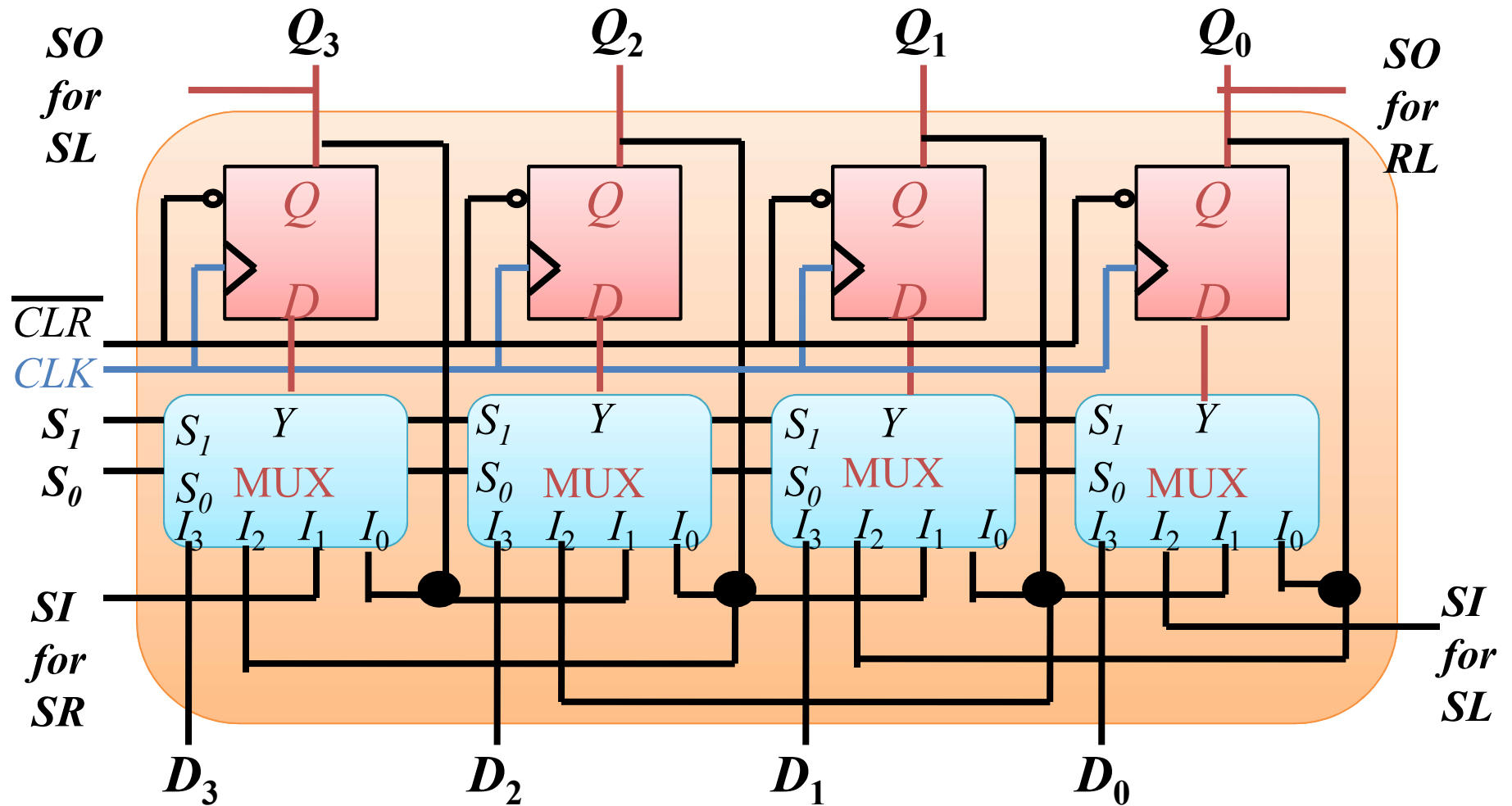
Universal Shift Register



Universal Shift Register

How to design
such an USR?

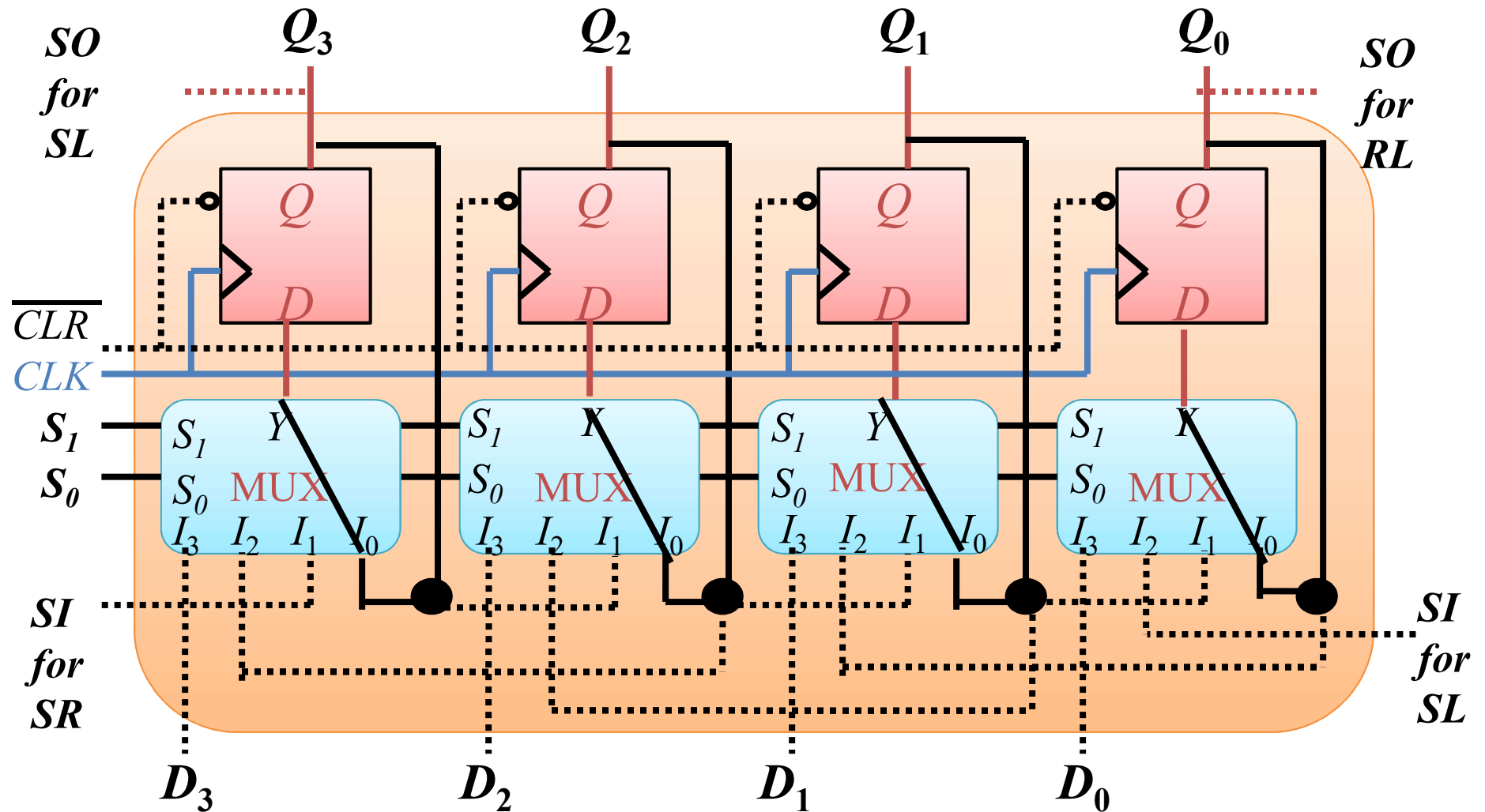
Universal Shift Register



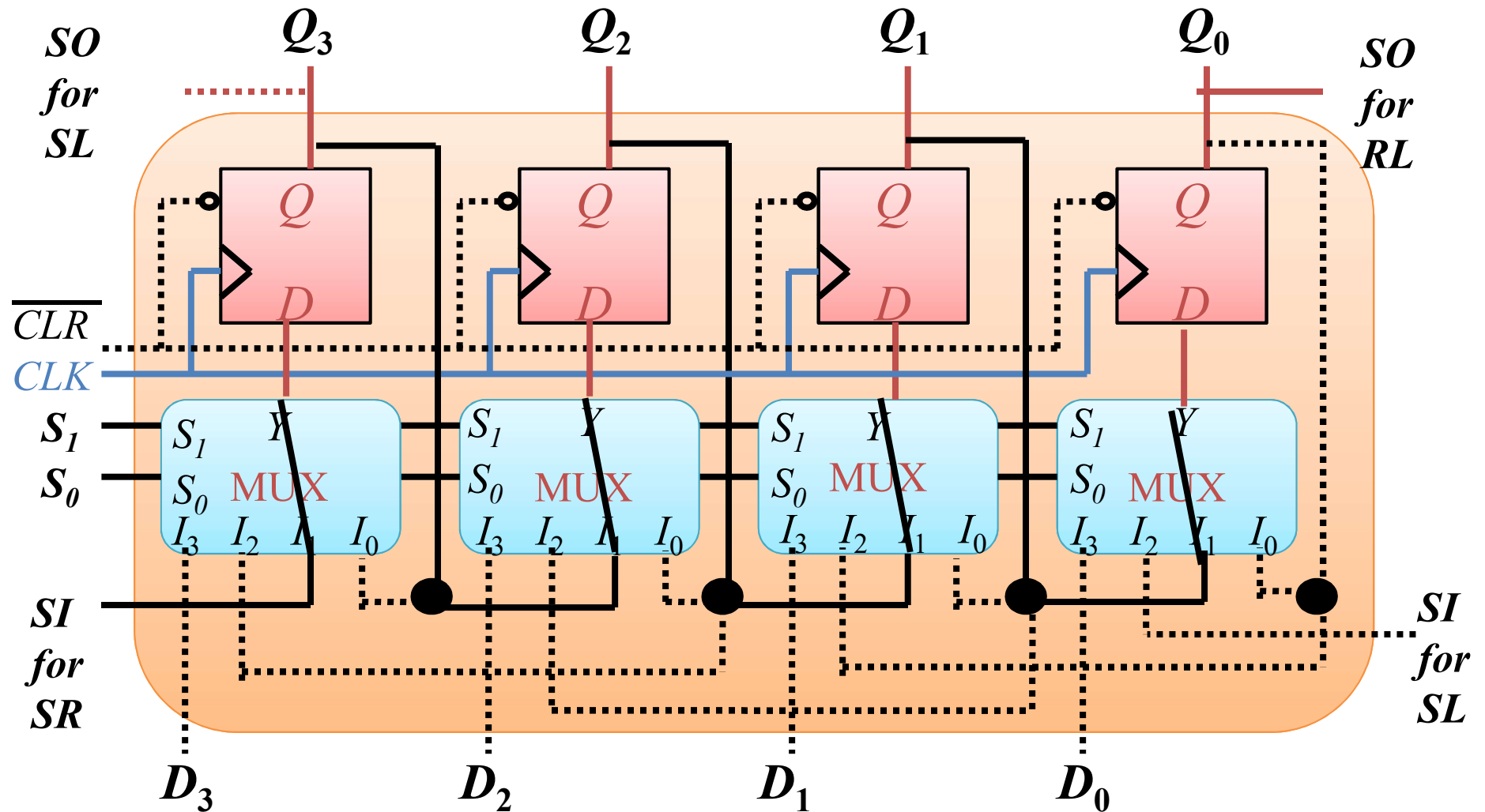
Operation Table for USR

S1	S0	Register Operation
0	0	No Change
0	1	Shift Right
1	0	Shift Left
1	1	Parallel Load

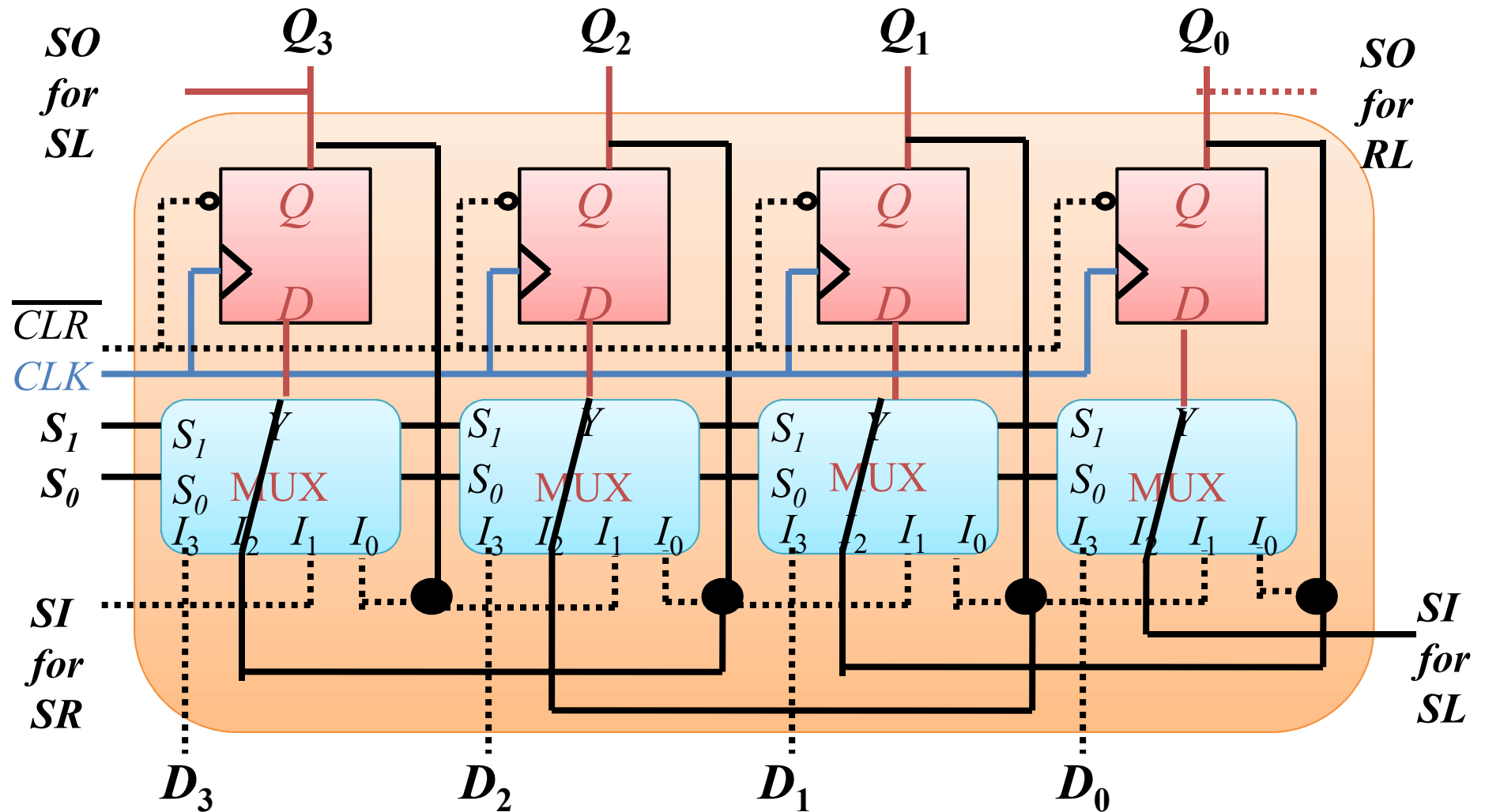
Universal Shift Register: $S_1S_0=00$



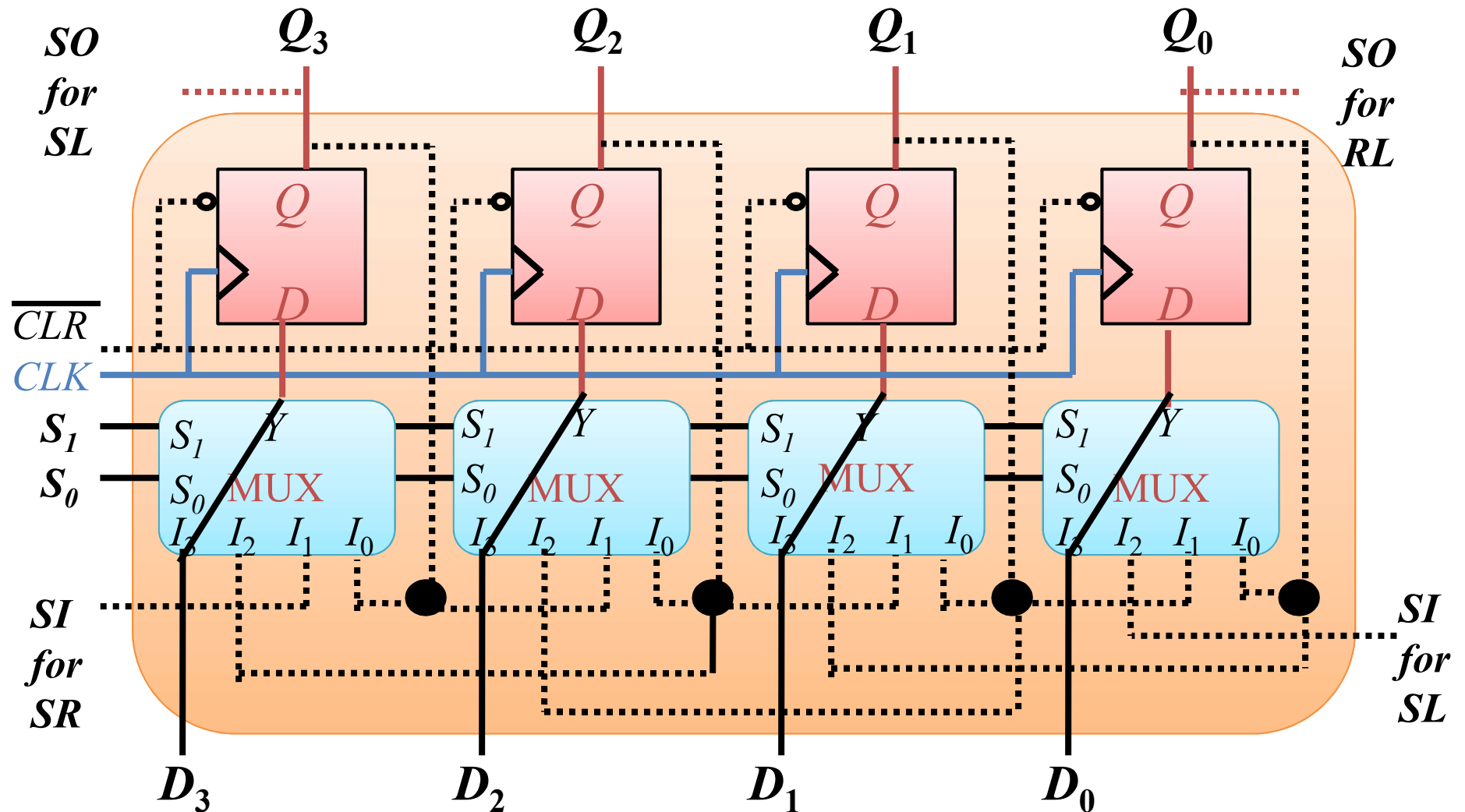
Universal Shift Register: $S_1S_0=01$



Universal Shift Register: $S_1S_0=10$



Universal Shift Register: $S_1S_0=11$



Verilog Code for Register

```
module universal_register (  
    input wire [3:0] data_input,  
    input wire [1:0] Sel,  
    input wire clock,  
    input wire left_in, input wire right_in,  
    output reg [3:0] data_output  
);
```


Verilog Code for Register

```
reg [3:0] reg_data;  
always @(posedge clock) begin  
    case(Sel)  
        2'b00: reg_data <= reg_data; // Retain  
        2'b01: reg_data <= {right_in, reg_data[3:1]}; // Rshift  
        2'b10: reg_data <= {reg_data[2:0], left_in}; // Lshift  
        2'b11: reg_data <= data_input; // Parallel load  
    endcase  
end  
always @(posedge clock) begin  
    data_output <= reg_data;  
end  
endmodule
```

S1	S0	Register Operation
0	0	No Change
0	1	Shift Right
1	0	Shift Left
1	1	Parallel Load

Design of Multifunction Register

- M function N bit Register
 - Last example : 4 functions and 4 bits register
- N number of $\text{ceil}[\text{Log}_2 M] \times 2^{\text{ceil}[\text{Log}_2 M]}$ size Multiplexor (Mux)
 - For 4 function 2x4 Mux
 - For 5 function 3x8 Mux
- Extra functions of Mux need to shunted (ignored) to do no work
 - For 5 function need a 3x8 Mux, we need to shunt $8-5=3$ lines shunted (ignored) to do no work

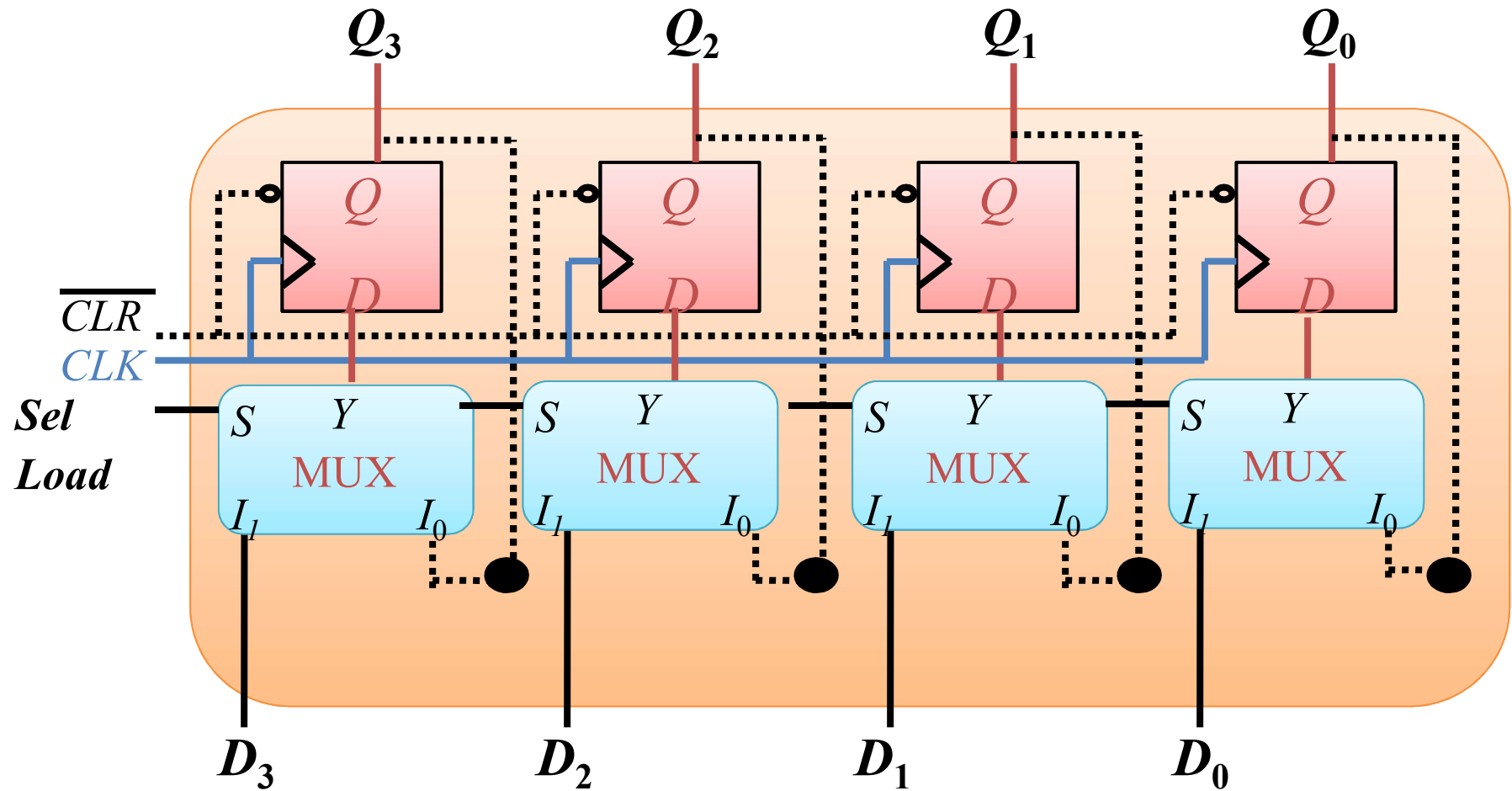
Memory

(SRAM: using Register)

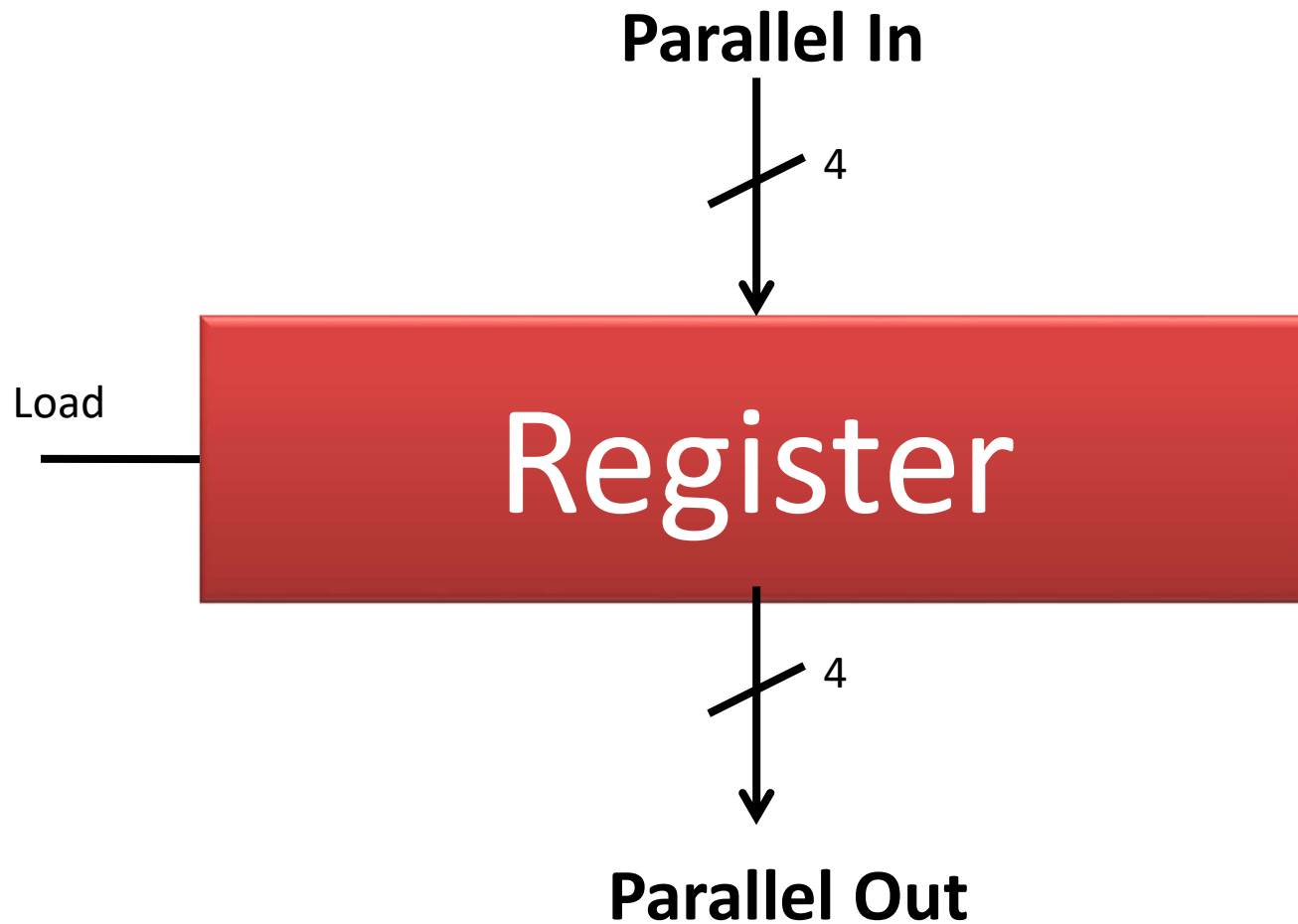
Memory

- Universal register : One register with many functionality
- Many registers clubbed with less functionality
 - No Shift
- **Parallel load (In) or Parallel Out to a specific register**
- Multiplexors used for selection of registers

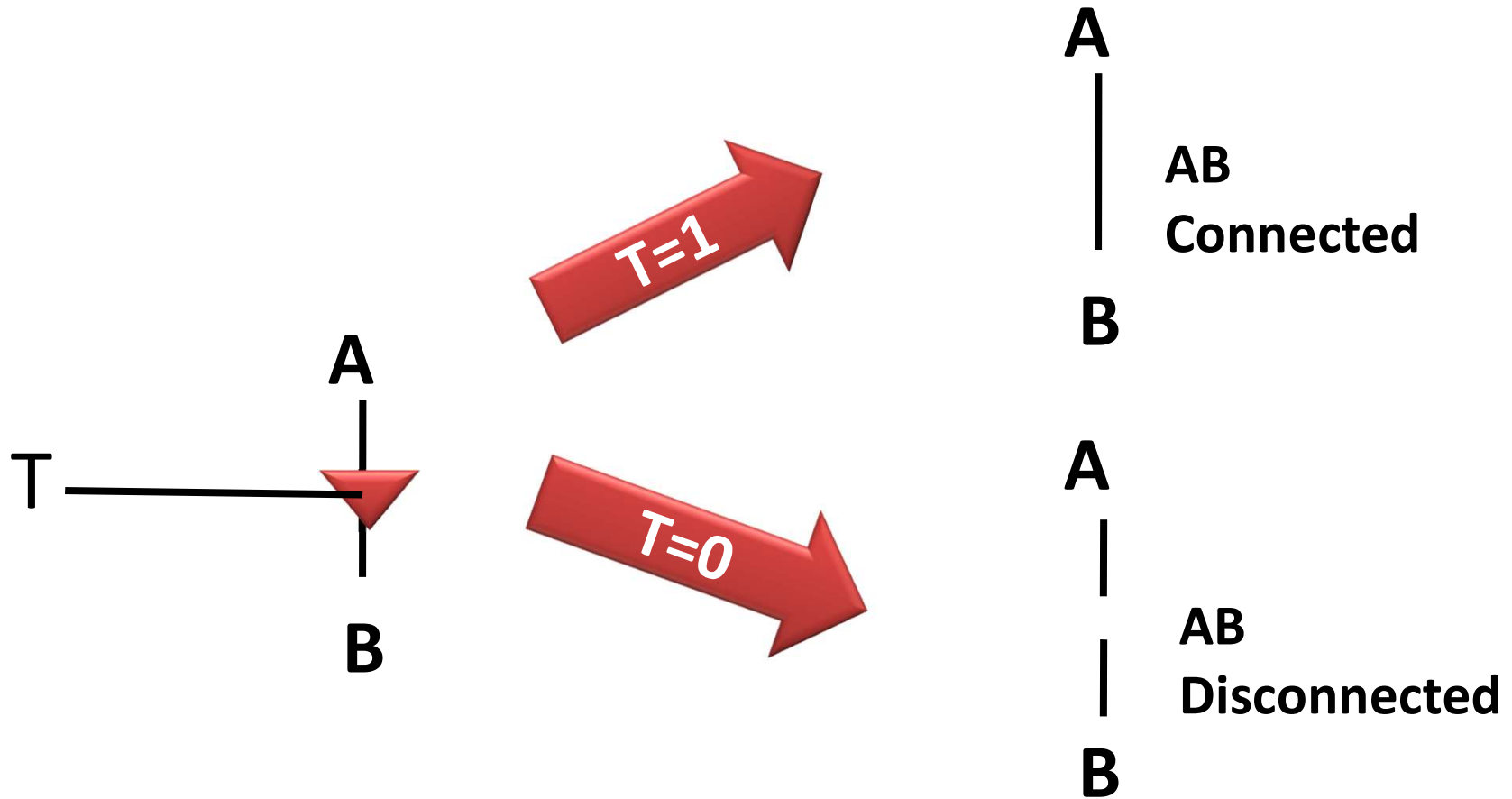
Register (PIPO)



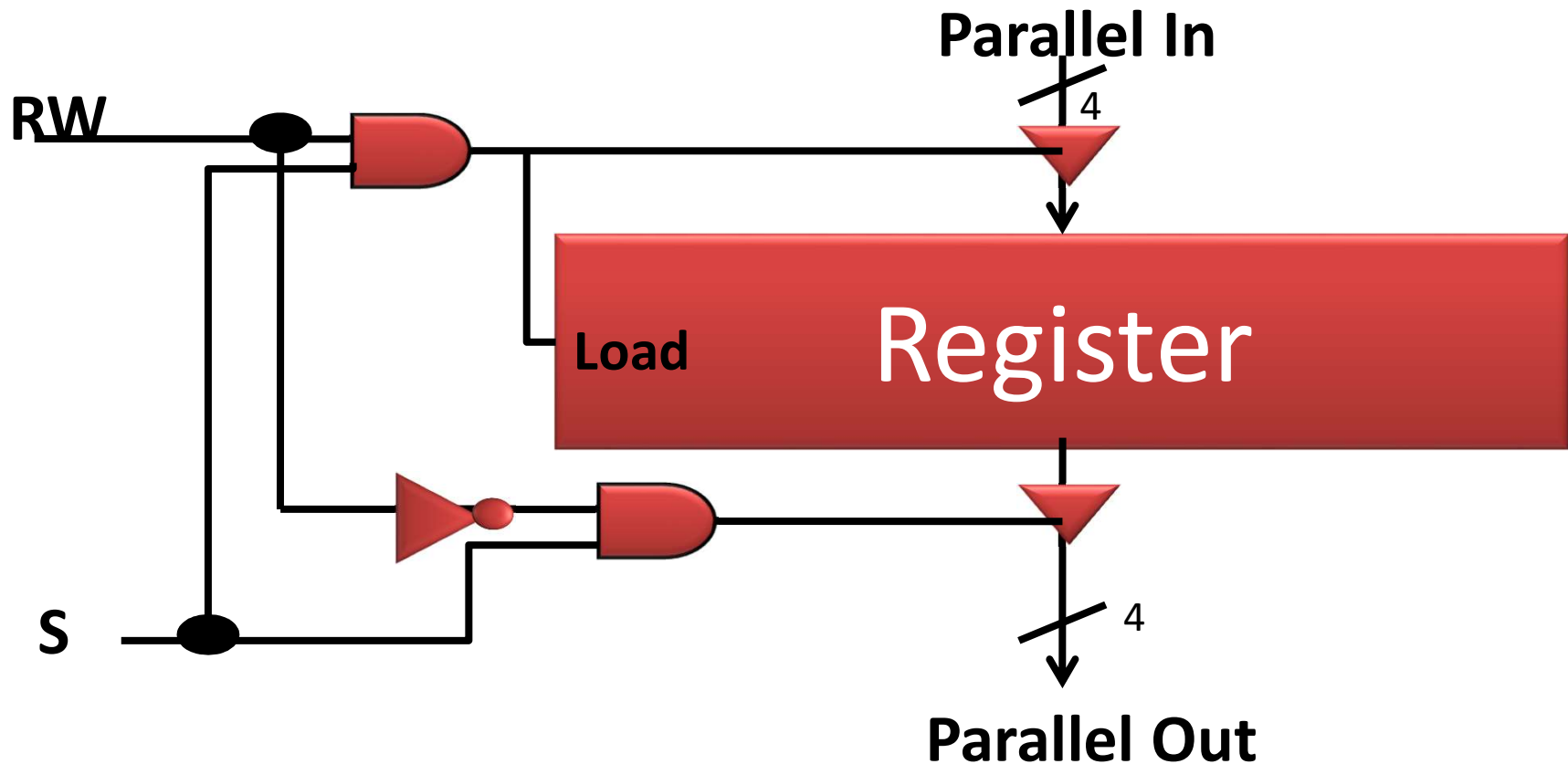
Register(PIPO)



Tri-state Buffer



Register with Read and write control

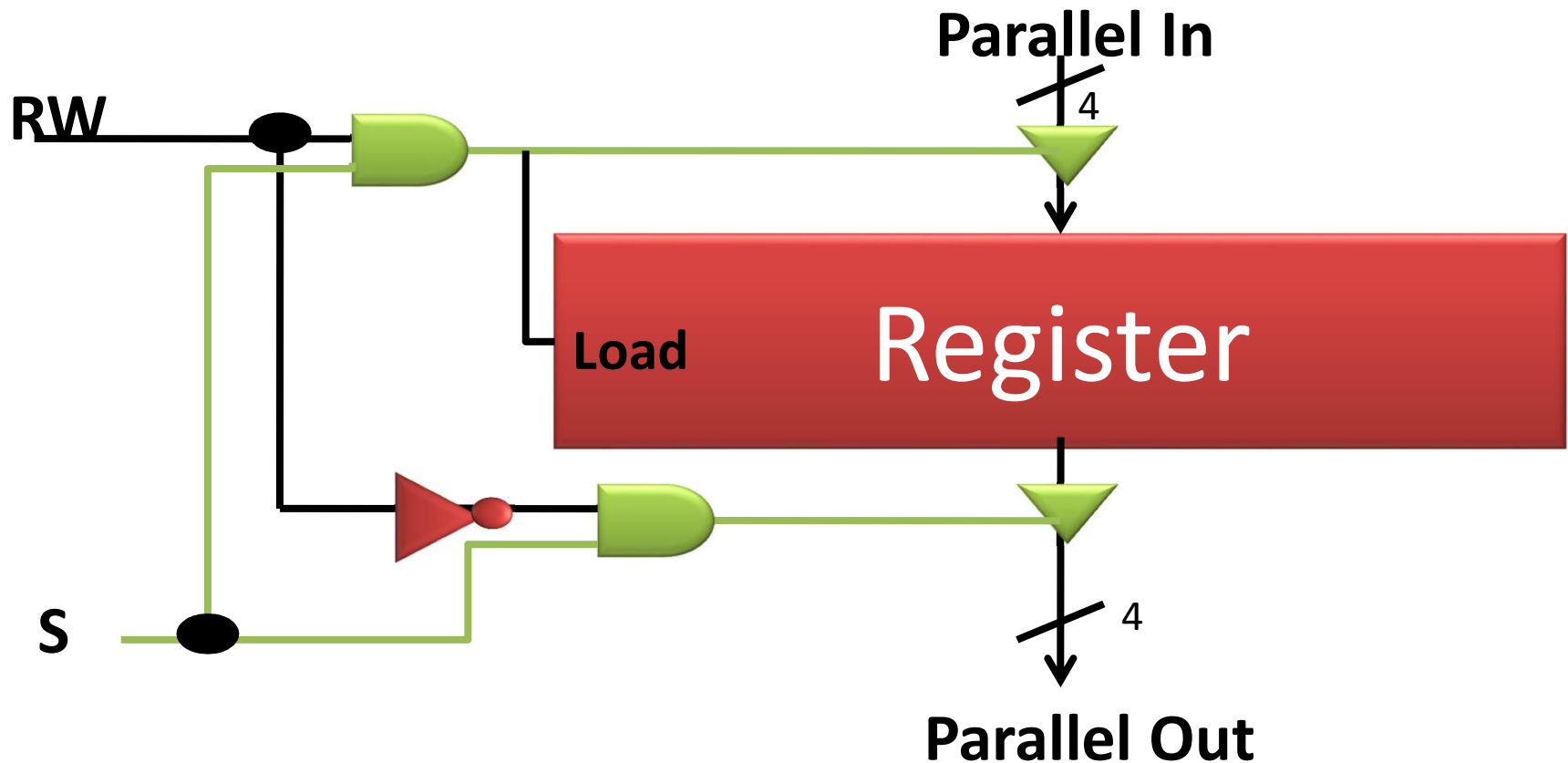


If $S = 0$, TS1 and TS2 will be open

$S=1$, $RW=1$, Write to Register 4 bit of data

$S=1$, $RW=0$, Read data from Register

Register with Read and write control



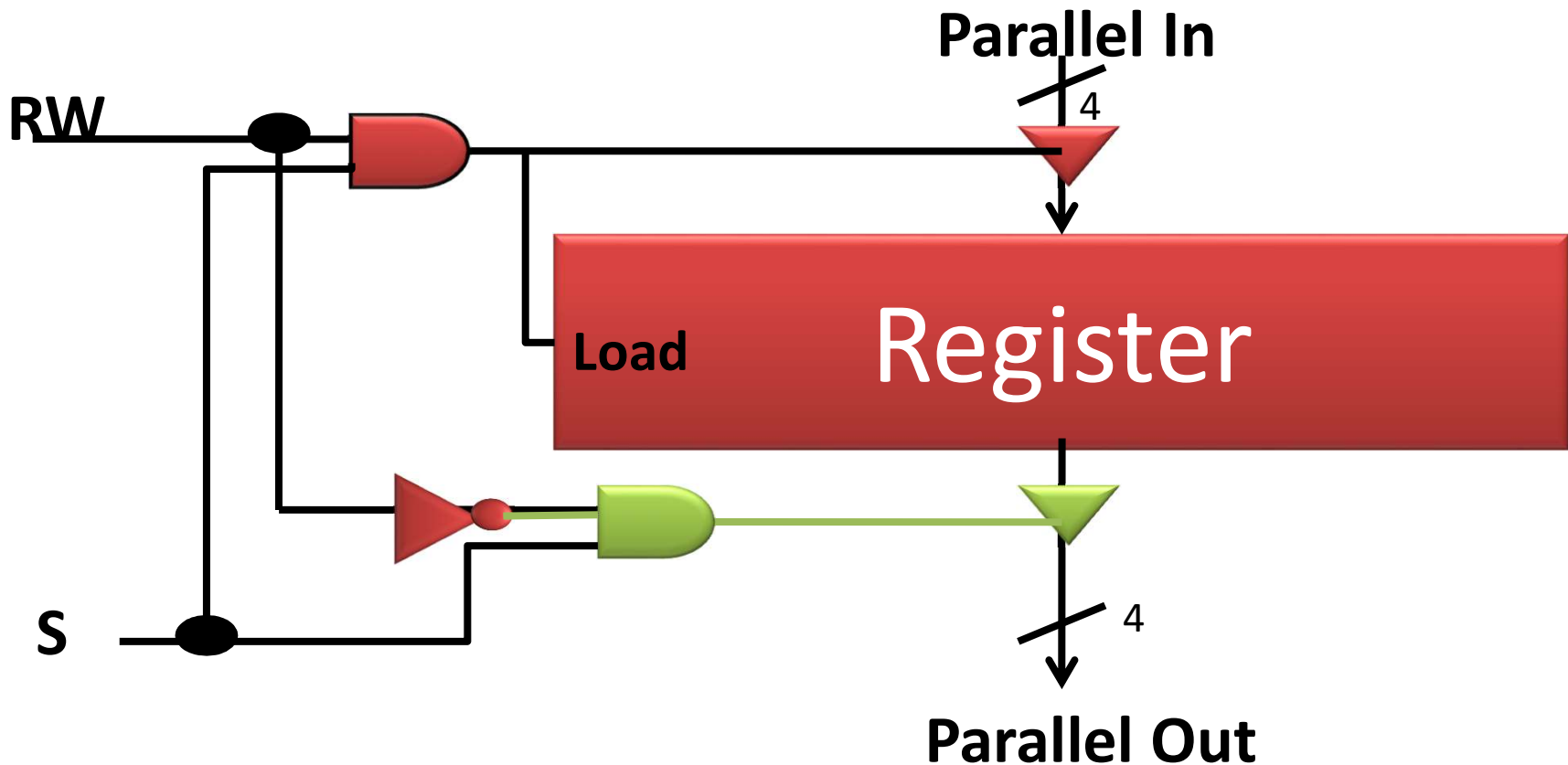
If $S = 0$, TS1 and TS2 will be open: not connected

$S=1, RW=1$, Write to Register 4 bit of data

$S=1, RW=0$, Read data from Register

Not selected for
Read or Write

Register with Read and write control



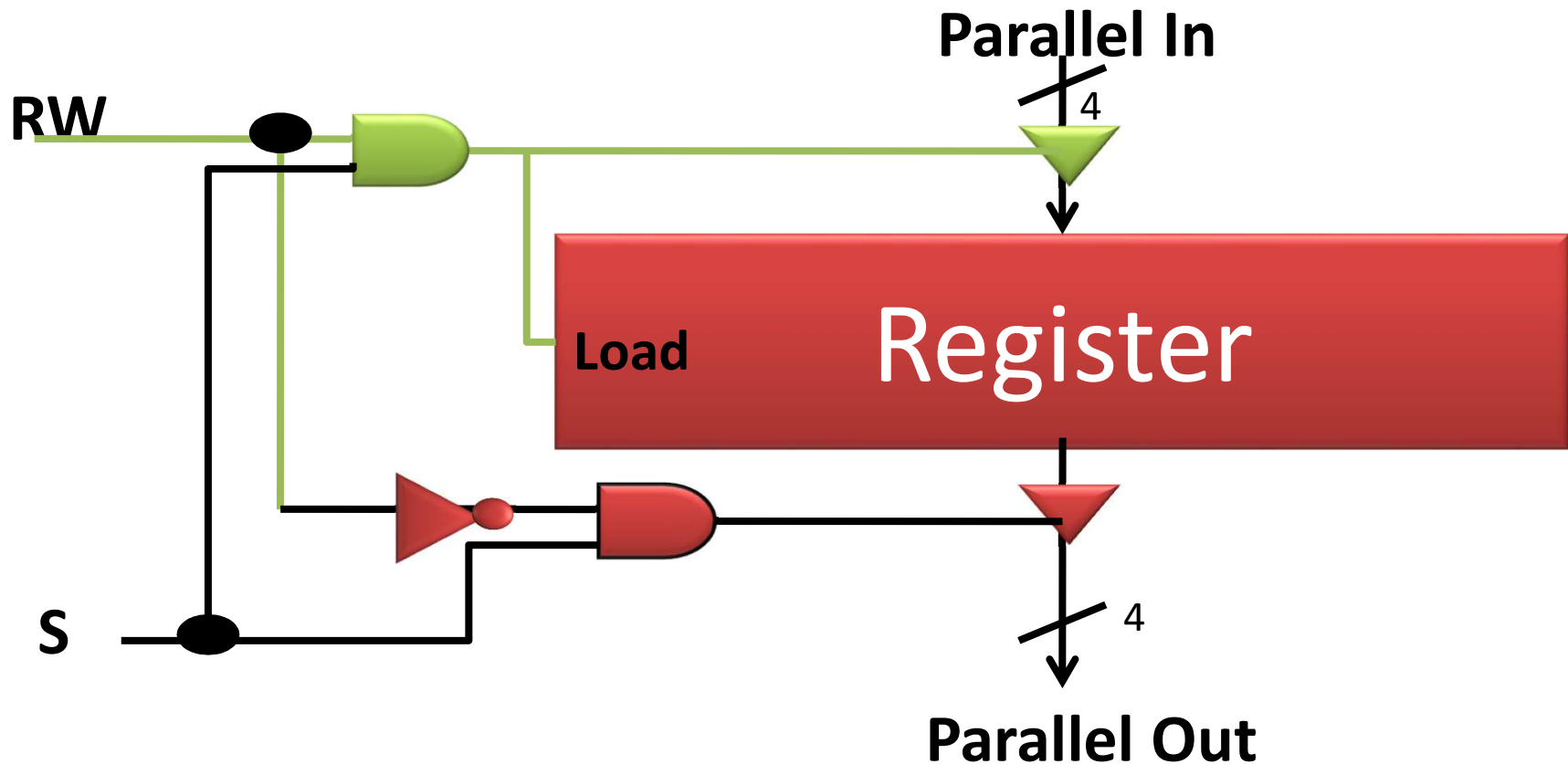
If $S = 0$, TS1 and TS2 will be open

$S=1, RW=1$, Write to Register 4 bit of data, only T1 connected

$S=1, RW=0$, Read data from Register

Selected for Write

Register with Read and write control



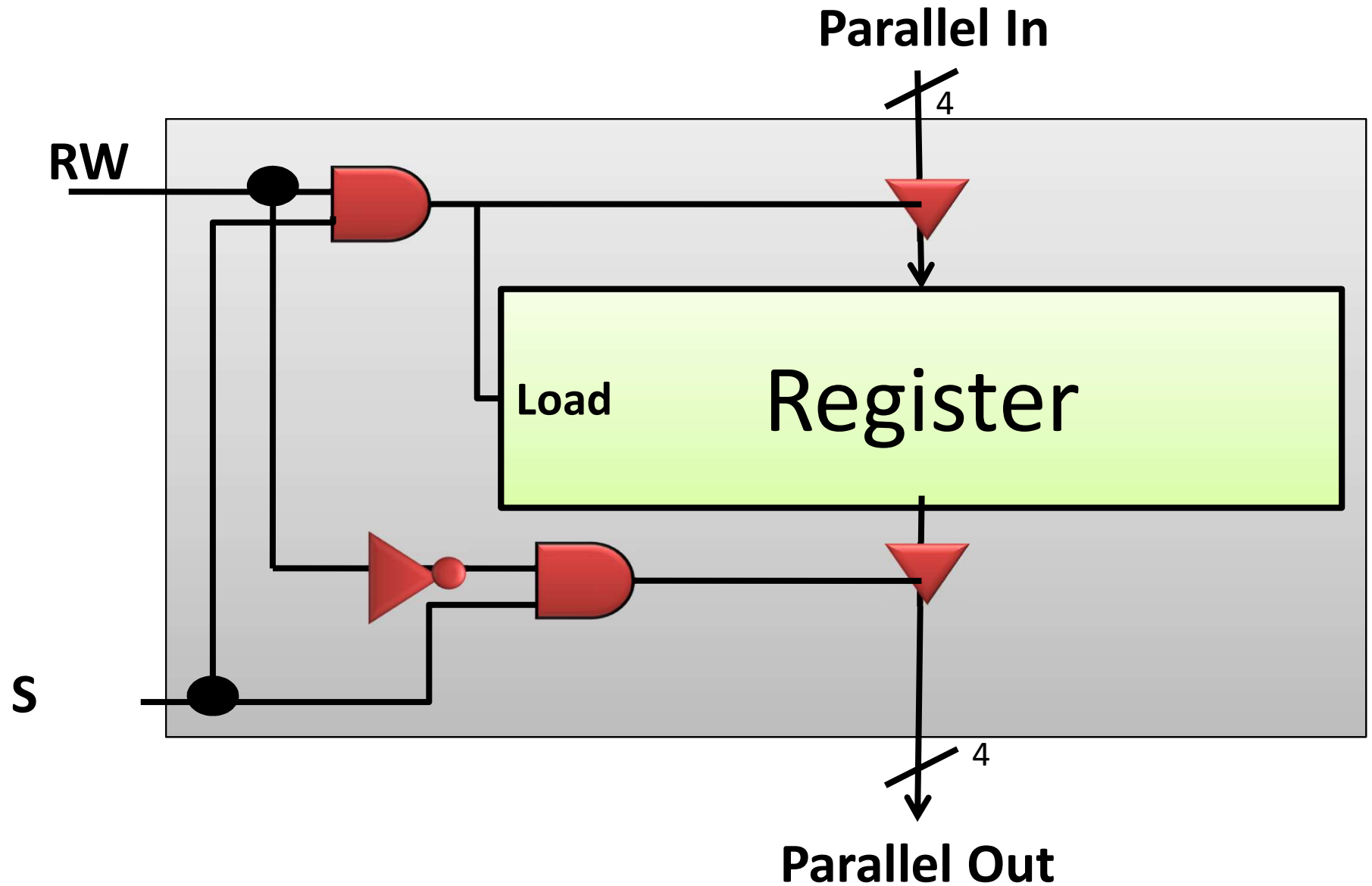
If $S = 0$, TS1 and TS2 will be open

$S=1$, $RW=1$, Write to Register 4 bit of data

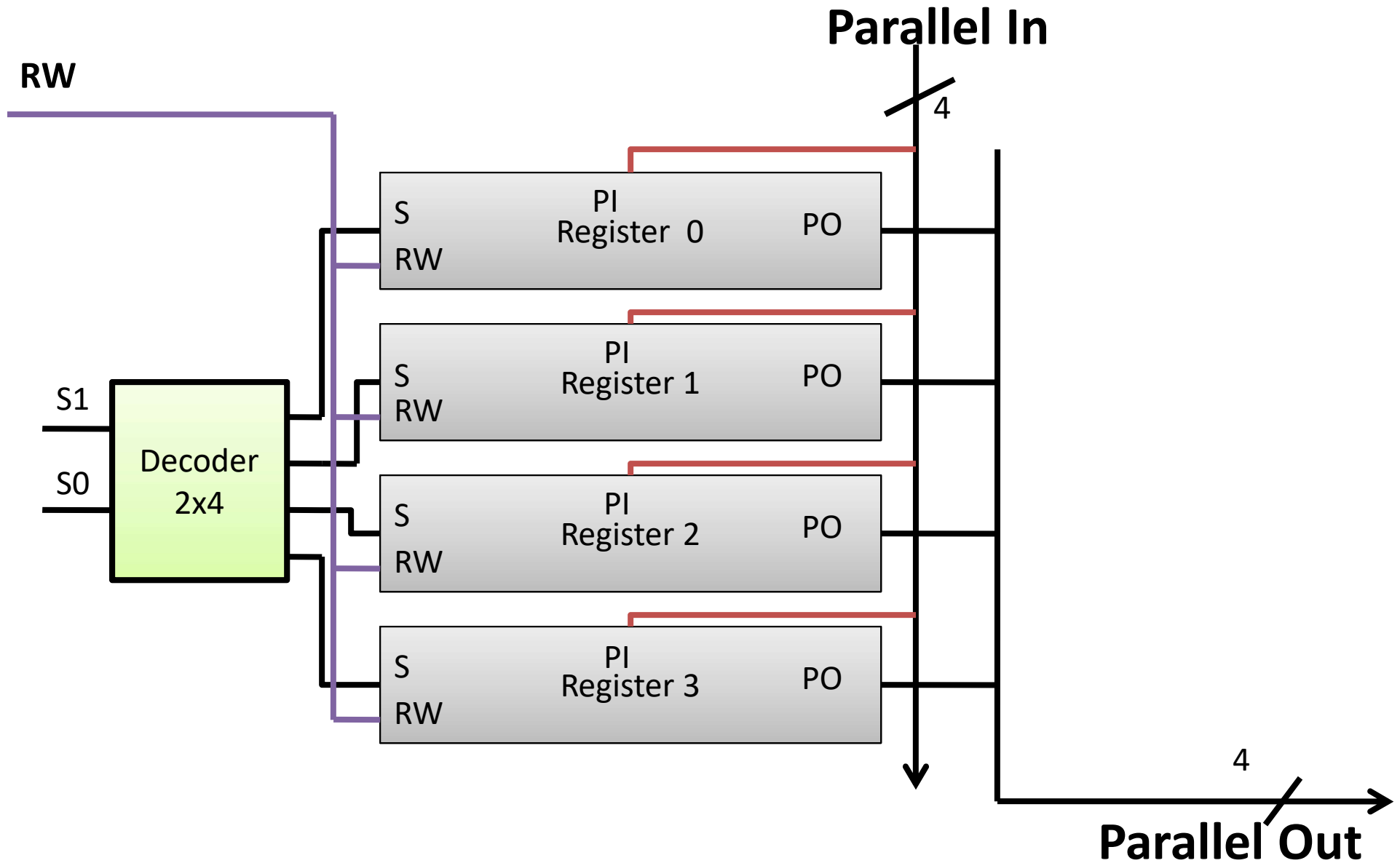
$S=1$, $RW=0$, Read data from Register

Selected for Read

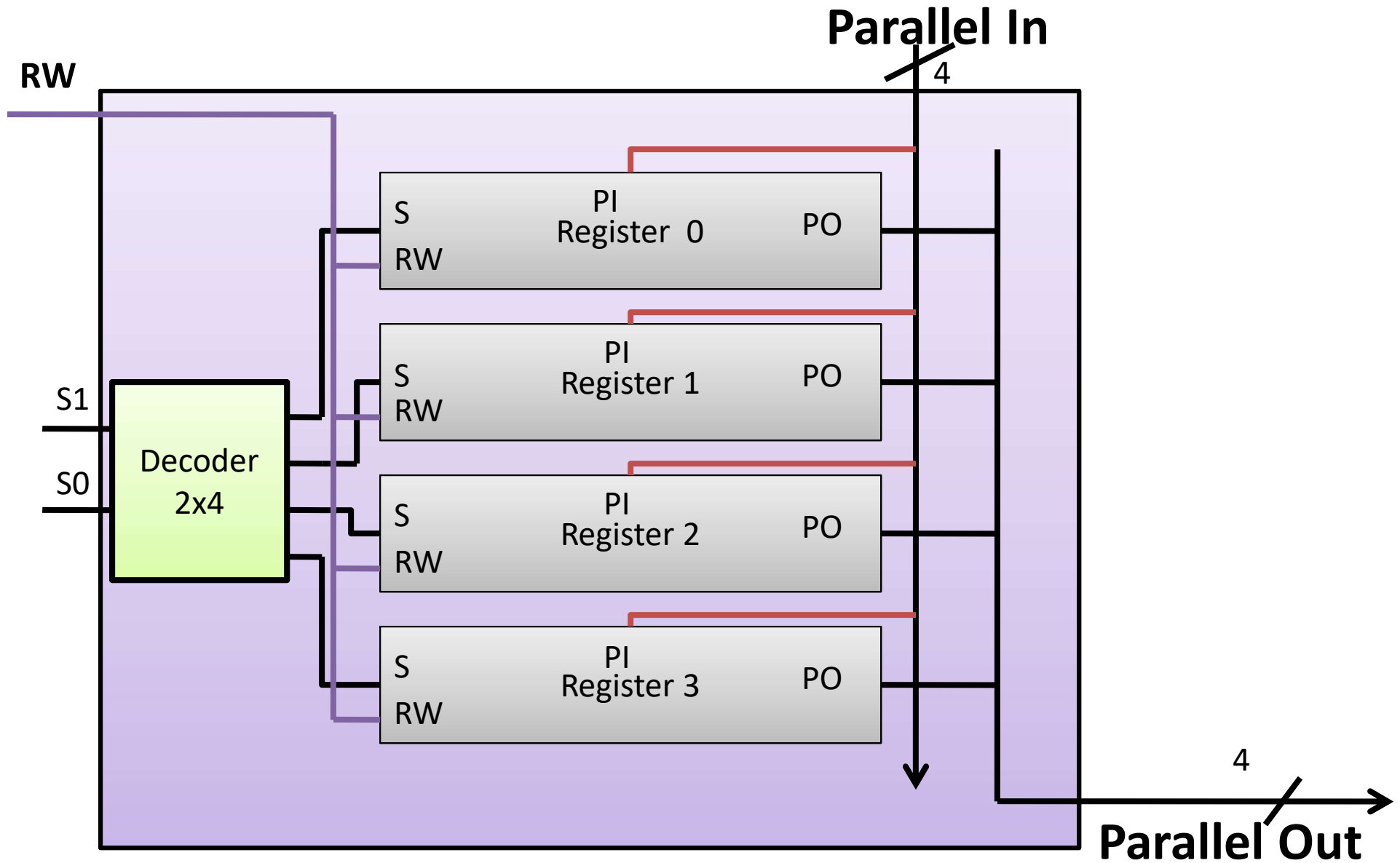
Register with Read and write control



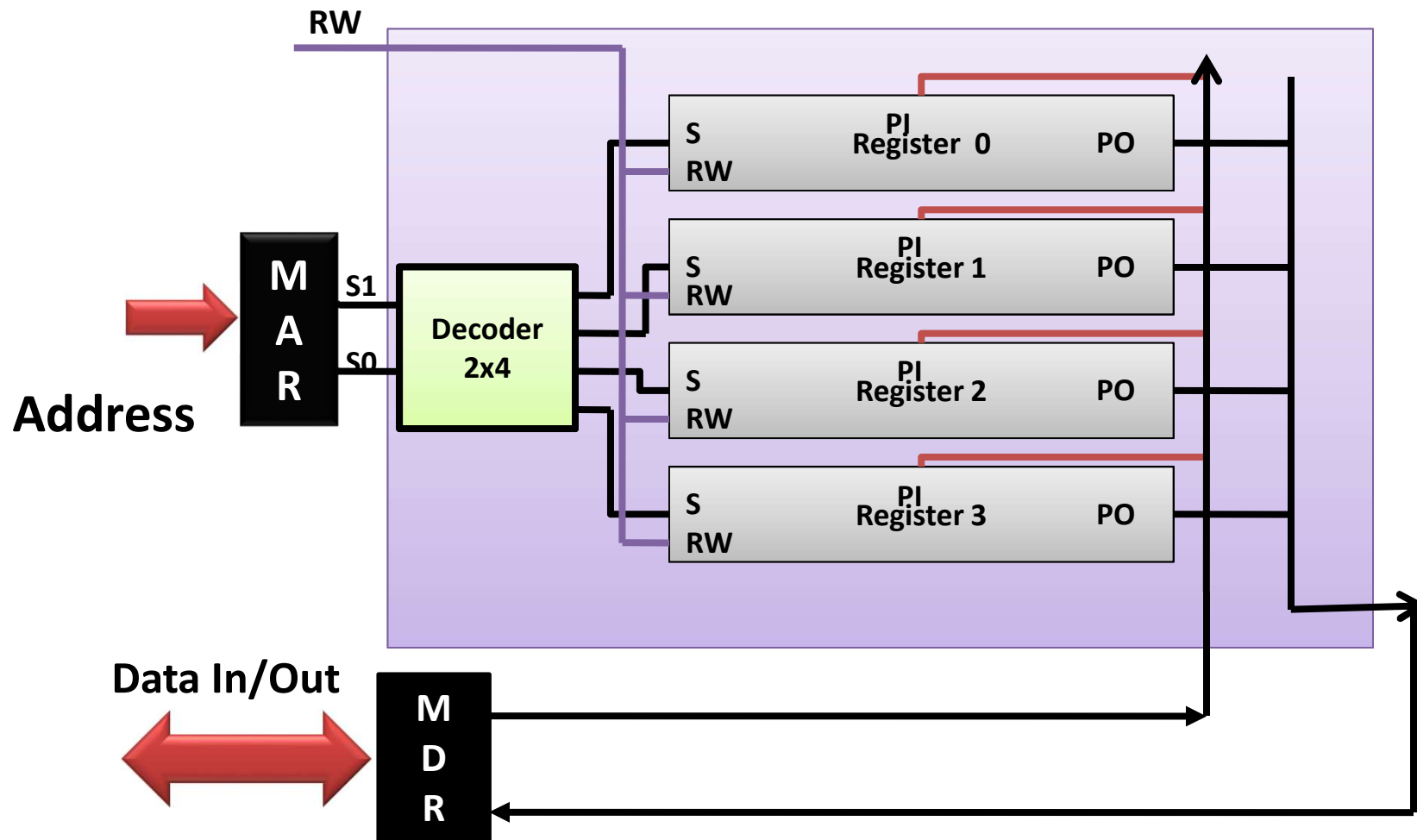
Many Register with RW control



Many Register with RW control



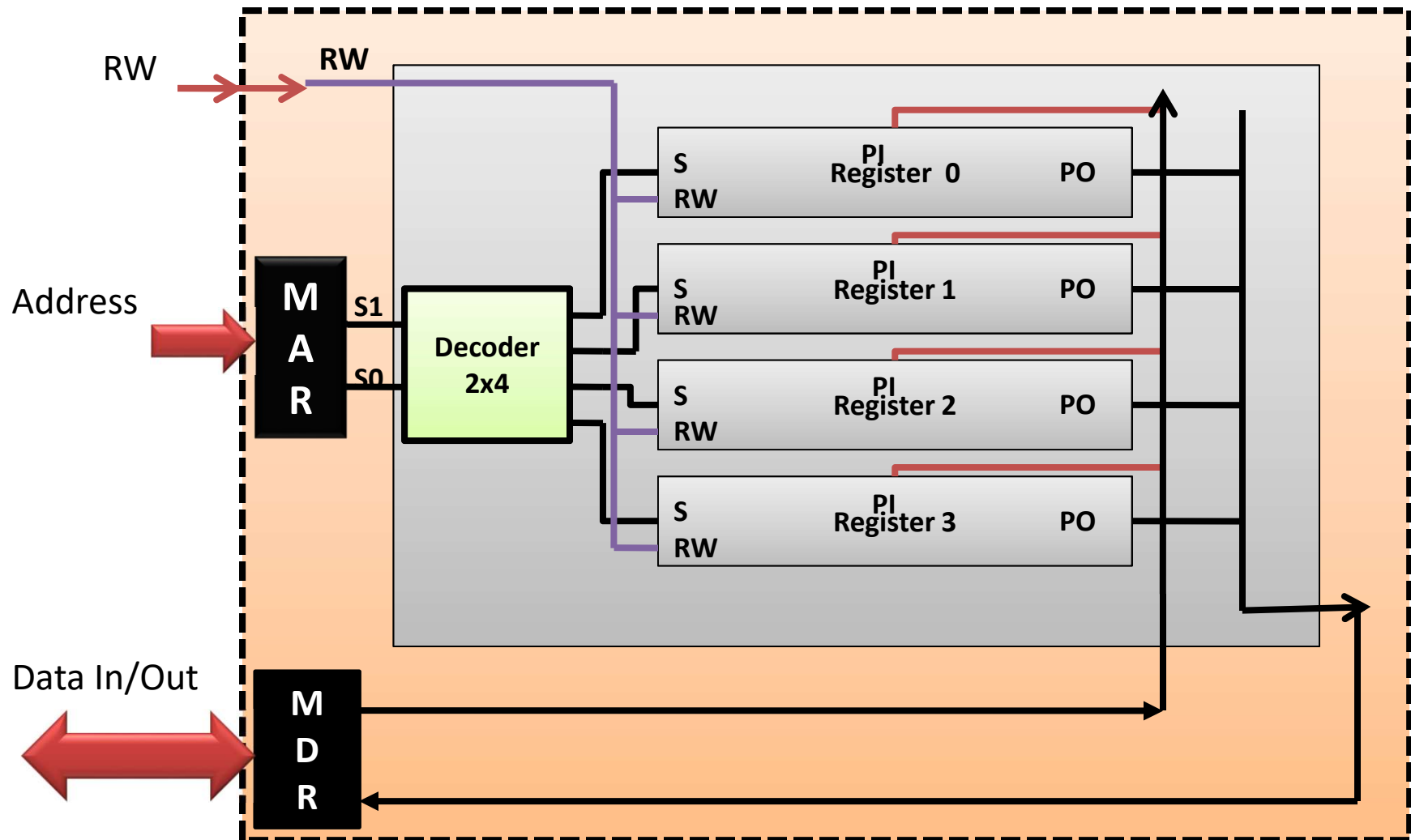
Memory with MAR and MDR



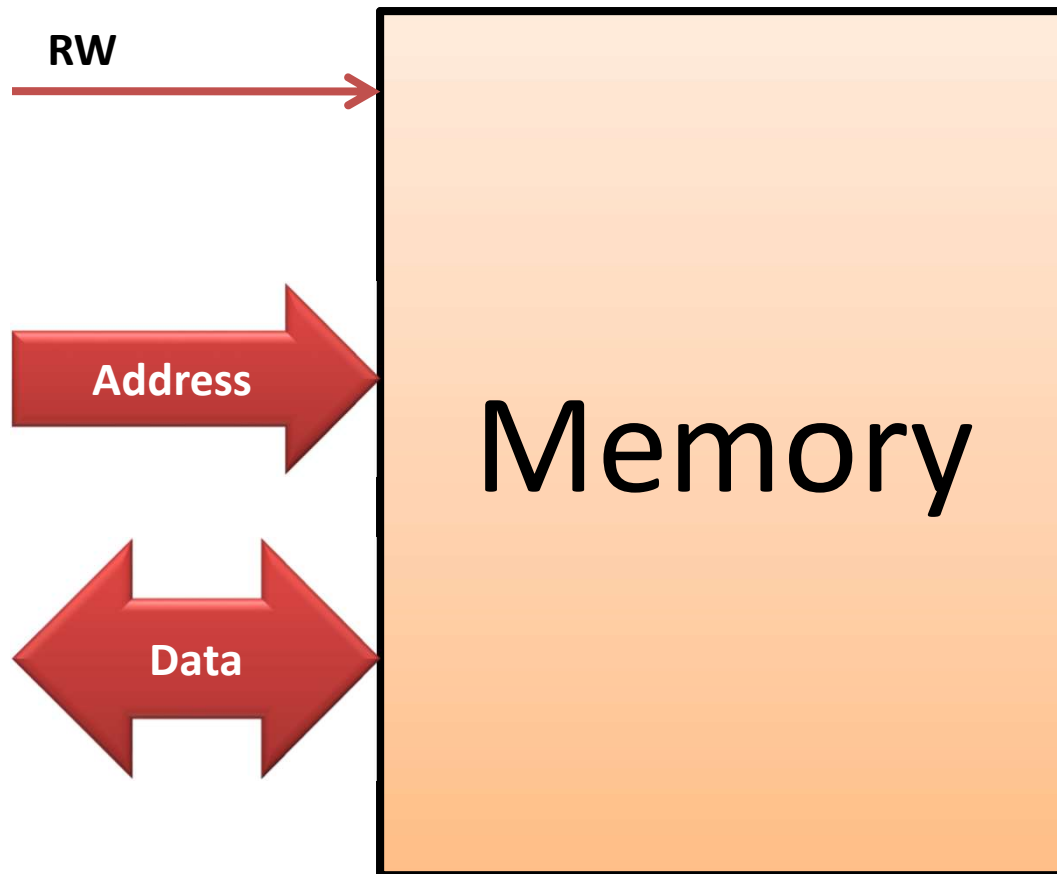
MAR: memory Address Register

MDR: Memory Data Register

Many Register with RW control: 4 four bit word Memory



General Memory



Memory : Locations & Word size

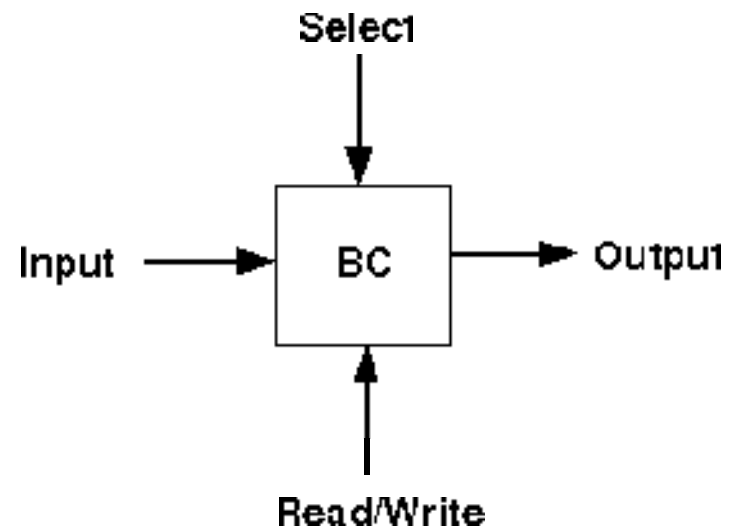
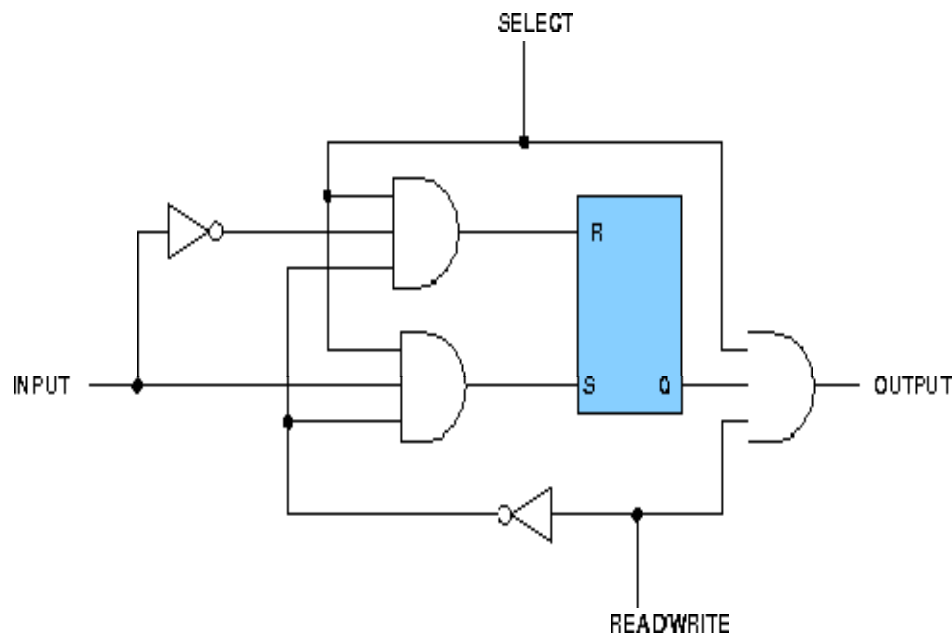
- ADDRESS decides the number of memory location
- MDR size : decides the WordSize
- Number of registers : $2 + 2^{\text{AddressSize}}$
 - 1 for MBR, 1 for MAR
- $\text{NumFF} = \text{MARsize} + \text{MDRsize} + \text{NumLoc} * \text{MDRSize}$

Memory : Asynchronous

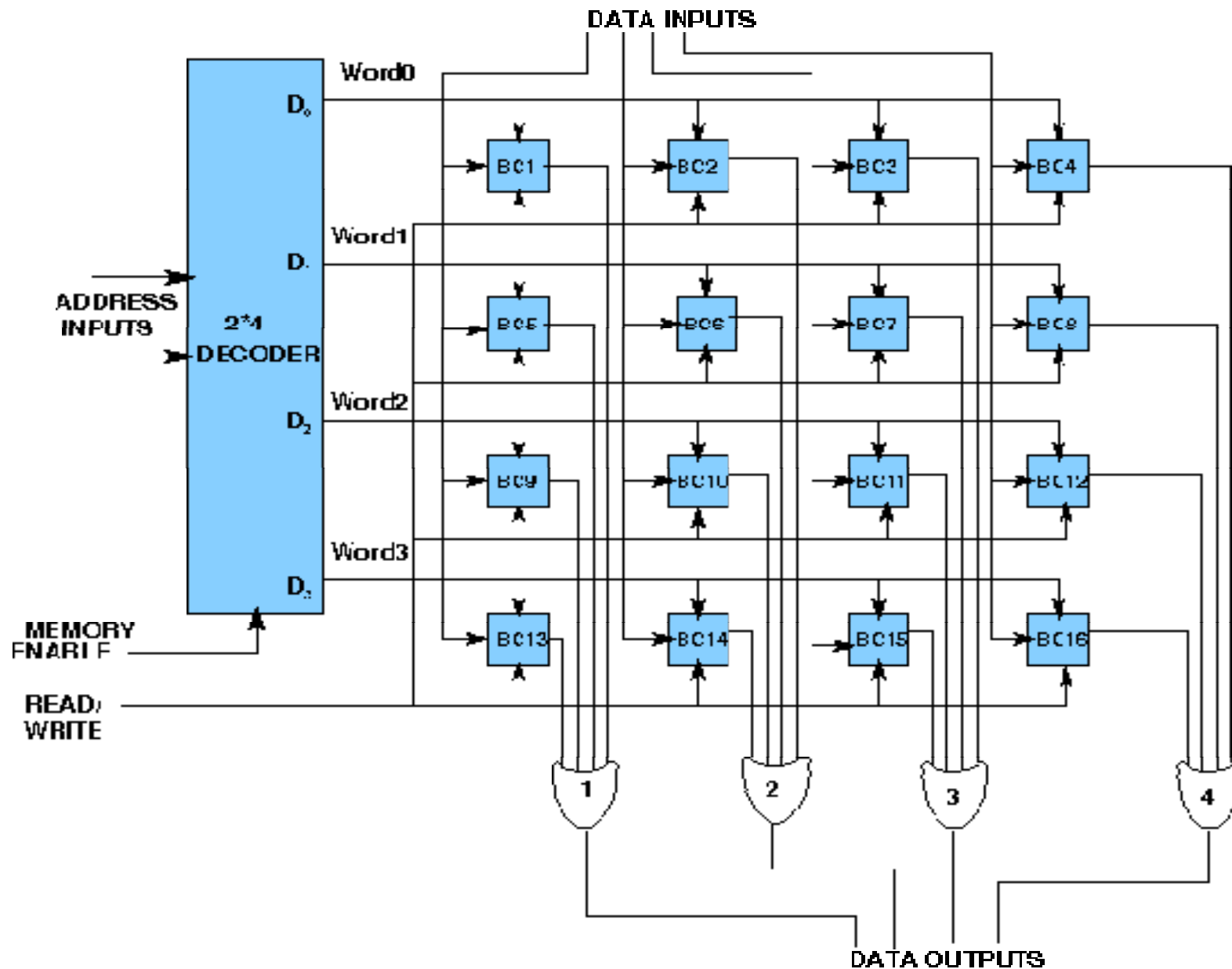
- Sync: When memory small and work with CPU Clock
 - Synchronous element is FF
- Async: When CPU clock is different, Mem is slower
 - RS Latch based Memory element Design

Memory : Asynchronous/Latch Based

RS Latch based Memory element Design



Memory : Asynchronous/Latch Based



Thanks